

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 5 de Rust

Web, Concurrencia y Juegos

Recomendado a partir de los 19-20 años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. Introducción al Bloque Práctico: Rust en el Mundo Real	1
1.1. El Servidor Web: Rust como el motor de la Internet moderna	1
1.2. Concurrencia y Asincronía: Exprimir el hardware al 100%	1
1.3. Tu primer Videojuego interactivo: ¿Por qué usamos Macroquad?	1
2. El Universo Web	3
2.1. ¿Cómo funciona la Web? El protocolo HTTP	3
2.2. El Protocolo HTTP y Tu Primer Servidor	3
2.3. ¿Cómo viaja la información por la web?	3
2.4. Peticiones (Requests) y Respuestas (Responses)	4
2.5. Proyecto Práctico: Creando un Mini-Servidor Web desde cero en Rust	5
2.6. Servidor web real con control de errores	10
3. Concurrencia Segura y Asincronía en Rust	12
3.1. La Analogía de la Oficina de Correos	12
3.2. Mecanismos de Ejecución en Rust	13
3.3. La conexión entre Concurrencia y Asincronía: Múltiples tareas sobre pocos hilos .	13
4. Concurrencia	16
4.1. El Motor de la Concurrencia: Las Clausuras (Closures)	17
4.2. El problema de los hilos y la solución con move	17
4.3. Las formas de crear hilos	18
4.3.1. A: El hilo independiente (thread::spawn con clausura)	19
4.3.2. B: Hilos con Ámbito o Concurrencia Estructurada (thread::scope)	20
4.4. Compartir Datos Mutables: El Poder del Mutex	22
4.5. Los Canales MPSC: pasar Mensajes en lugar de compartir Memoria	25
5. Asincronia: async / await en Rust	28
5.1. Ejemplo práctico con tokio	28
5.2. Ventajas frente a la concurrencia con Hilos del Sistema Operativo (OS Threads) .	29
5.3. El Rol del Runtime (Ej. Tokio o el de Macroquad)	30
5.4. ¿Por qué llamamos a Macroquad Runtime interno?	30
6. Proyecto Final de Ciclo - Videojuego Gráfico Macroquad	32
6.1. Proyecto Guiado: Nuestro primer juego gráfico (Clon de Pong)	33
6.1.1. Añadir un sistema de puntuación al código	37
6.1.2. Integr un canal mpsc y un hilo secundario para registrar los puntos en segundo plano	40
6.2. Ejercicios propuestos	44
6.3. Desafío propuesto para el alumno (Personalización e integración)	44
7. Servidor web asíncrono multihilo con Tokio	45
7.1. Mecanismos clave que intervienen en el programa	46
7.2. ¿Cómo reparte Tokio las tareas entre los hilos? (El Planificador)	47
7.2.1. Diferencia de Consumo de Memoria: std::thread vs tokio::spawn	48
8. Los retos propuestos	48
8.1. 🏆 Reto Final A: El Panel de Control del Operador	49
8.2. 🏆 Reto Final B: El Sistema de Alertas Inteligente	49

1. Introducción al Bloque Práctico: Rust en el Mundo Real

¡Bienvenido al último paso de tu viaje con Rust! En los cuadernos anteriores has aprendido a controlar la memoria mediante el sistema de propiedad (**ownership**), a estructurar tus datos con elegancia y a gestionar errores como un profesional de la ingeniería de software. Has estado trabajando en un entorno controlado, asegurándote de que los cimientos de tu conocimiento sean indestructibles.

Sin embargo, la teoría sin práctica es letra muerta. Este cuaderno final no está aquí para enseñarte más reglas sintácticas, sino para **aplicar todo lo aprendido en tres proyectos reales** que interactúan con el mundo exterior. Pasaremos de ejecutar líneas en la terminal a construir aplicaciones palpables. A continuación, te explicamos por qué hemos elegido estos tres pilares para cerrar tu formación:

1.1. El Servidor Web: Rust como el motor de la Internet moderna

Cuando navegas por internet, das por hecho que las páginas web responden al instante. Detrás de esa magia hay servidores que procesan miles de peticiones por segundo. Tradicionalmente, esto se hacía con lenguajes que consumían mucha memoria o que eran propensos a fallos de seguridad.

En este bloque aprenderás a construir un servidor web desde cero. Verás cómo el estricto control de tipos de Rust y su velocidad nativa lo convierten en la tecnología favorita de empresas como Discord o Cloudflare para sostener la infraestructura del mundo digital. Entenderás cómo hacer para recibir datos de un usuario, procesarlos y devolver una respuesta en microsegundos de forma totalmente segura.

1.2. Concurrencia y Asincronía: Exprimir el hardware al 100%

Ya lo adelantamos en el cuaderno anterior: los procesadores actuales no son significativamente más rápidos que los de hace unos años, simplemente tienen más núcleos. Un programa síncrono tradicional es como tener un Ferrari y usarlo solo para ir a comprar el pan a 20 km/h; estás desperdiciando el potencial del silicio.

En este apartado romperemos las barreras del tiempo y el orden secuencial. Aprenderás a coordinar tareas para que tu programa haga «muchas cosas a la vez». Verás la diferencia exacta entre **concurrencia** (repartir tareas pesadas entre los núcleos de la CPU) y **asincronía** (aprovechar los tiempos muertos en los que el ordenador espera datos de la red o del disco duro). Y lo mejor de todo: descubrirás por qué la comunidad llama a Rust el rey de la «**Fearless Concurrency**» (Concurrencia sin miedo), ya que su compilador te impedirá cometer los típicos errores de memoria que vuelven locos a los programadores en otros lenguajes.

1.3. Tu primer Videojuego interactivo: ¿Por qué usamos Macroquad?

Un videojuego es la prueba de fuego definitiva para cualquier lenguaje de programación. ¿Por qué? Porque un juego no puede permitirse «frenar» para limpiar la basura de la memoria; necesita renderizar gráficos, calcular físicas y leer el teclado del jugador exactamente **60 veces por segundo** (60 FPS) sin un solo frenazo.

Para lograr esto de forma divertida y didáctica, utilizaremos **Macroquad**. Es una librería de Rust (**crate**) espectacular porque elimina toda la complejidad matemática de bajo nivel de las tarjetas gráficas (OpenGL/Vulkan) y te ofrece una interfaz limpia y directa. Te permite dibujar pantallas, mover personajes y detectar colisiones escribiendo pocas líneas de código. Macroquad nos permite centrarnos en la lógica pura del juego utilizando el rendimiento bruto y la seguridad que Rust ya nos da de serie.

El objetivo de este cuaderno:

Demostrarte que Rust no es solo un lenguaje académico para evitar errores de memoria. Es una herramienta ultrapotente diseñada para construir el software del futuro: rápido, eficiente, concurrente y, por qué no, también divertido. ¡Empecemos!

2. El Universo Web

El desarrollo moderno de software no se puede entender sin su conexión con la red y, es aquí donde el protocolo HTTP se consolida como el idioma universal para la comunicación entre clientes y servidores. A lo largo de este capítulo, exploraremos los fundamentos de cómo funciona la Web y cómo viaja la información a través de la infraestructura global de internet. Analizaremos en detalle la anatomía de las peticiones (Requests) y respuestas (Responses) que componen cada interacción digital, para finalmente consolidar todo este conocimiento teórico en un proyecto práctico de ingeniería: la construcción y programación de un mini-servidor web desde cero utilizando Rust, aprovechando su velocidad y seguridad de memoria nativas.

2.1. ¿Cómo funciona la Web? El protocolo HTTP

Estamos acostumbrados a viajar por la web de forma intuitiva colocando una dirección en el campo de URL del navegador o haciendo clic en los enlaces de las páginas web. Sin embargo, cada vez que entras en una página web, tu navegador (cliente) envía una Petición (Request) a un ordenador remoto (servidor). El servidor lee esa petición y devuelve una Respuesta (Response) que suele contener texto en formato HTML, imágenes o datos.

El protocolo HTTP es el idioma estándar en el que se comunican cliente y servidor. Una petición básica le dice al servidor: «Quiero obtener (GET) el archivo principal (<https://www.google.com/>)». El servidor responde con un código de estado —como 200 OK si todo fue bien, o el famoso 404 Not Found si no existe el archivo— seguido del contenido de la página solicitada.

2.2. El Protocolo HTTP y Tu Primer Servidor

El protocolo HTTP (Hypertext Transfer Protocol) es la base de la comunicación en la World Wide Web, permitiendo que un cliente (como un navegador) y un servidor intercambien datos de forma estructurada. En el ecosistema de Rust, crear un primer servidor web implica **abrir un puerto TCP** para escuchar conexiones entrantes, **procesar los bytes recibidos** según las reglas de HTTP y **devolver una respuesta** adecuada. Para proyectos iniciales, esto se puede lograr utilizando la biblioteca estándar (`std::net::TcpListener`), lo que permite entender los fundamentos de la red antes de dar el salto a frameworks (marcos de trabajo) más avanzados del ecosistema como **Axum** o **Actix-web**.

2.3. ¿Cómo viaja la información por la web?

La información en la web viaja empaquetada en mensajes de texto plano a través de la pila de protocolos TCP/IP, donde HTTP opera en la capa de aplicación para definir el formato de dichos mensajes. Cuando un usuario introduce una URL, el navegador realiza una resolución DNS para encontrar la dirección IP del servidor, establece una conexión segura mediante TLS/SSL (HTTPS) y transmite la información fragmentada en paquetes de red. Al ser HTTP un protocolo sin estado (stateless), cada interacción se trata de forma independiente, lo que significa que el servidor no recuerda las peticiones anteriores a menos que se utilicen mecanismos adicionales como cookies o tokens de sesión.

2.4. Peticiones (Requests) y Respuestas (Responses)

Una petición (Request) HTTP es el mensaje que envía el cliente e incluye obligatoriamente un método o verbo de los siguientes (GET, POST, PUT, DELETE), una URL o ruta, la versión del protocolo, cabeceras (headers) con metadatos y, opcionalmente, un cuerpo (body) con datos.

Por tanto, cuando escribes una dirección web en el navegador o pinchas sobre un enlace, estás mandando al servidor una petición (Request) similar a la siguiente:

```
GET /index.html HTTP/1.1
Host: ://mi-servidor-rust.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html
Connection: close
```

Hemos supuesto que mandas la petición al servidor: `mi-servidor-rust.com`

- Línea 1: Indica la acción (GET, para obtener datos), el archivo solicitado (/index.html) y la versión del protocolo (HTTP/1.1).
- Líneas 2-5: Son las cabeceras (metadatos). Le dicen al servidor a qué dominio se conecta (Host), qué navegador se usa (User-Agent) y qué tipo de archivo espera recibir (Accept).

Por otro lado, la respuesta (Response) es la contestación del servidor, la cual contiene un código de estado numérico (como 200 OK, 404 Not Found o 500 Internal Server Error), cabeceras que describen el contenido (como Content-Type) y el cuerpo que alberga el recurso solicitado (HTML, JSON, imágenes). En Rust, estos componentes se modelan mediante estructuras de datos y enums que garantizan un tipado seguro, evitando errores comunes al parsear o construir mensajes web.

En el caso de la petición anterior, la respuesta del servidor podría ser la siguiente:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 114
Connection: close

<html>
  <body>
    <h1>¡Hola Mundo desde Rust!</h1>
    <p>Tu primer servidor funciona.</p>
  </body>
</html>
```

- Línea 1: La línea de estado. Confirma que todo ha ido bien devolviendo el código numérico 200 OK.
- Líneas 2-4: Las cabeceras de respuesta. Informan al navegador que lo que va a leer es un archivo web (text/html) y cuánto mide exactamente en bytes (Content-Length).
- Línea 5: Una línea en blanco obligatoria. Es el separador crítico que le dice al navegador: «Fin de los metadatos, empieza la página web».

- Bloque final: El cuerpo (Body). El código HTML puro que el navegador interpretará para pintarlo de forma visual en la pantalla del usuario.

2.5. Proyecto Práctico: Creando un Mini-Servidor Web desde cero en Rust

Para construir un mini-servidor web nativo en Rust sin dependencias externas (crates), lo que llamamos de forma nativa, se utiliza el módulo `std::net::TcpListener` para escuchar conexiones en nuestra máquina y responder con una página web real. También nos permitirá enlazar el programa a una dirección de nuestra máquina local y un puerto, como por ejemplo **127.0.0.1:8080** que también puede escribirse como **localhost:8080**.

El servidor se diseña mediante un bucle continuo que acepta **el flujo de datos entrante (TcpStream)**, lee los bytes de la petición que llegan en ese flujo mediante un **buffer** de memoria y los procesa para identificar la ruta solicitada, es decir, el recurso solicitado en la petición.

Nota: Un **stream** es una secuencia de datos que van llegando uno a uno a lo largo del tiempo. Por ejemplo, cuando visualizamos una película en YouTube recibimos los datos mediante stream.

Un **buffer** de memoria es un espacio de almacenamiento temporal donde se guardan datos de forma provisional mientras se transfieren de un lugar a otro. Funciona como una sala de espera o una caja de cartón donde el programa va acumulando los bytes que llegan desde el exterior (como una petición web) hasta que tiene la cantidad suficiente para procesarlos todos juntos de forma eficiente.

Finalmente, el programa evalúa la solicitud y utiliza el método `stream.write_all()` para enviar de vuelta una cadena de texto con formato analizable por el navegador, completando así el ciclo de vida de una transacción web con la máxima eficiencia y seguridad de memoria que caracteriza a Rust.

Código de ejemplo

- Abre una terminal integrada de VS Code en la carpeta **proyectos-rust** (o en cualquier otra) y crea el proyecto de Rust **servidor_web** con la orden:

```
cargo new servidor_web
```

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado, borrando previamente todo lo que hubiera escrito en ese fichero.

(Después del listado hay una explicación del código)

Proyecto: **servidor_web**

src/main.rs

```
use std::io::{prelude::*, BufReader};
use std::net::{TcpListener, TcpStream};
```

```

fn main() {
    // 1. Escuchamos en la dirección local (localhost) en el puerto 8080
    // El unwrap() aquí asume que el puerto 8080 está libre en el sistema.
    let listener = TcpListener::bind("127.0.0.1:8080").unwrap();
    println!("Servidor web iniciado en http://127.0.0.1:8080");

    // 2. Esperamos y procesamos las conexiones de los clientes
    // listener.incoming() es un iterador que devuelve un
    // Result<TcpStream, Error> por cada intento de conexión.
    for stream in listener.incoming() {
        // Asumimos que la conexión de red no falló al establecerse
        let stream = stream.unwrap();
        gestionar_conexion(stream);
    }
}

fn gestionar_conexion(mut stream: TcpStream) {
    // Creamos un lector con búfer en memoria RAM
    // para no saturar el hardware de red
    let buf_reader = BufReader::new(&mut stream);

    // Leemos únicamente la primera línea de la petición
    // (ej. "GET / HTTP/1.1")
    // El primer unwrap() abre el Option del iterador;
    // el segundo abre el Result de la lectura técnica.
    let linea_peticion = buf_reader.lines().next().unwrap().unwrap();
    println!("Petición recibida: {}", linea_peticion);

    // Definimos la respuesta HTTP con código 200 OK y contenido HTML
    let contenido_html = r#"
        <!DOCTYPE html>
        <html lang="es">
        <head>
            <meta charset="UTF-8">
            <title>Mi Servidor en Rust</title>
            <style>
                body { font-family: sans-serif; background-color: #f4f4f9;
text-align: center; padding-top: 50px; }
                h1 { color: #df4b32; }
            </style>
        </head>
        <body>
            <h1>¡Hola desde mi servidor Rust!</h1>
            <p>Este servidor ha sido programado desde cero en 2º de
Bachillerato.</p>
        </body>
        </html>
    "#;

    let longitud = contenido_html.len();
    // Construimos la estructura exacta que exige el protocolo HTTP/1.1
    let respuesta = format!(
        "HTTP/1.1 200 OK\r\nContent-Length: {}\r\n\r\n{}",
        longitud, contenido_html
    );
};

```

```
// Enviamos la respuesta en bytes de vuelta a la tubería del cliente
stream.write_all(respuesta.as_bytes()).unwrap();
}
```

Explicación del código

1) En el main, la siguiente línea:

```
// 1. Escuchamos en la dirección local (localhost) en el puerto 8080
let listener = TcpListener::bind("127.0.0.1:8080").unwrap();
```

Crea el servidor (listener) y queda escuchando en la dirección local del ordenador (127.0.0.1) y el puerto 8080. Habría sido equivalente escribir "localhost:8080". Solo podrás lanzar peticiones al servidor desde tu propio ordenador utilizando la dirección y puerto.

Nota técnica: Usamos `.unwrap()` aquí porque si el puerto 8080 ya estuviese ocupado por otra aplicación en tu ordenador, Rust detendría el programa inmediatamente con un pánico (panic!) de forma segura, avisándote de que el puerto no está disponible.

2) El bucle for:

```
// 2. Esperamos y procesamos las conexiones de los clientes
for stream in listener.incoming() {
    let stream = stream.unwrap();
    gestionar_conexion(stream);
}
```

2.1) ¿Que es un stream?

A modo de recordatorio:

Un **stream** (o flujo de datos) es como una tubería digital continua que conecta dos puntos (por ejemplo, tu navegador y tu servidor en Rust) para transportar información de manera constante, bit a bit y en un orden estricto.

La analogía de **YouTube**: Cuando ves un vídeo en **streaming**, no esperas a que se descargue la película entera de 2 GB para empezar a verla; el vídeo te va llegando como un flujo constante de datos que reproduces en tiempo real mientras el resto sigue viajando por la red.

2.2) ¿Qué resulta ser `listener.incoming()`?

`listener.incoming()` es un iterador (una secuencia infinita) que se queda «escuchando» el puerto de red. Imagínalo como una cinta transportadora en una fábrica que nunca se apaga.

- **Cómo funciona en la práctica:** El bucle `for` detiene el programa y se queda esperando mirando a la cinta. En el momento en que un cliente (un navegador) se conecta, la cinta se mueve y nos entrega un paquete individual. Ese paquete es la variable **stream**: una tubería de comunicación directa y exclusiva entre ese cliente específico y nuestro servidor. A través de esa tubería (usando herramientas como un **buffer**), podremos leer lo que el cliente nos pide y responderle. Una vez terminamos, el bucle `for` vuelve a mirar a la cinta a la espera del siguiente paquete.

Los tres puntos claves de la analogía:

1. El Iterador (`incoming()`) = La cinta transportadora (el mecanismo que los trae).
2. El bucle `for` = El operario que se queda pausado esperando a que llegue algo por la cinta.
3. El stream = El paquete físico (la tubería) que el operario agarra de la cinta. Cada cliente tiene su propia tubería independiente.

El truco de Rust: Este iterador nunca termina por sí solo. Mantiene el servidor vivo y funcionando en un bucle continuo para recibir visitas una detrás de otra.

2.3 Las sentencias `unwrap()`

1) ¿En qué se traduce `let stream = stream.unwrap()` en nuestro programa?

Esta línea realiza dos acciones fundamentales de Rust: desempaquetar el resultado —que aquí suponemos seguro— y redefinir la variable.

A. El desempaquetado con `stream.unwrap()`

En Rust, una conexión de red puede fallar en el último milisegundo (por ejemplo, si el cliente pierde la conexión justo cuando el servidor la estaba aceptando debido a un fallo de cable o caída de red). Por eso, el iterador no te da la conexión directamente, sino que te da un envoltorio de seguridad llamado **`Result<TcpStream, Error>`**.

`.unwrap()` significa: «Confío en que la conexión está bien. Abre el envoltorio, dame el flujo de datos real (`TcpStream`) para que pueda trabajar con él y, si hubo un error crítico, detén el programa».

Es decir, si realmente ha habido un error en la conexión, tal como lo tenemos programado, el sistema fallaría y daría un pánico (`panic!`), provocando que el programa se colgara.

Lo hacemos así porque sabemos que tenemos una altísima probabilidad de que no haya fallos, de hecho, tanto nuestra petición como respuesta solo viajan por el interior de nuestra máquina, sin salir a la red. Esto nos permite simplificar mucho el programa al suponer que las cosas irán bien ahorrándonos todo el tratamiento de errores que si veremos en la siguiente versión de servidor web.

B. La redefinición: `let stream = stream.unwrap()`

Esto se conoce en Rust como Variable Shadowing (Sombreado de variables). Permite reutilizar el mismo nombre (`stream`) de una variable para crear otra nueva variable.

- **El primer `stream`** (el del `for`) es el envoltorio con el posible error que hemos supuesto que no se va a producir.
- **El segundo `stream`** (el del `let`) es la conexión limpia, extraída con el `unwrap()` y lista para usar en la función `gestionar_conexion()`.

2) la línea `stream.write_all(respuesta.as_bytes()).unwrap();`

Aquí, nada puede fallar en la conversión a bytes. Si el `unwrap()` salta y el programa se congela, la culpa es única y exclusivamente de la tarjeta de red (de `write_all`), jamás de `.as_bytes()`.

¿Por qué `.as_bytes()` nunca puede fallar?

- En Rust, un tipo `String` o `&str` está obligado por el compilador a ser siempre texto UTF-8 válido. Rust valida esto en el momento en que creas la variable. Como el texto ya está guardado internamente como bytes en la memoria del ordenador, `.as_bytes()` no hace ninguna operación matemática ni transformación: simplemente te da acceso directo a esos números. Al no haber procesamiento, esta función no devuelve un `Result`, devuelve los bytes directamente y tiene 0% de probabilidad de fallar.

¿Qué es entonces lo que puede hacer saltar el `unwrap()`? El peligro está en `stream.write_all()`. Esta función sí devuelve un `Result<(), std::io::Error>`. Los motivos físicos y reales por los que este método puede fallar y hacer que tu `unwrap()` provoque un pánico (`panic!`) en el servidor son:

- El cliente cerró la pestaña del navegador de golpe: Si el usuario hace clic en el enlace y, antes de que el servidor termine de enviarle los datos, cierra la pestaña o cancela la carga, la tubería (`stream`) se rompe. `write_all` intentará escribir en un cable vacío y dará un error de «Conexión abortada».
- Problemas físicos o de red local: Si la conexión del cliente pierde cobertura (si es por Wi-Fi), si se cae el cable de red o si el router local se satura justo en ese milisegundo.
- El sistema operativo se quedó sin recursos: El sistema se quedó sin memoria para gestionar los buffers de salida de los sockets de red (un caso muy extremo, pero técnicamente posible).

3) La función `gestionar_conexion()`

La función `gestionar_conexion()` es el cerebro operativo de nuestro servidor. Es la encargada de recibir la «tubería» (`stream`) que abrió el cliente, leer lo que nos están pidiendo y, escribir la respuesta HTTP de vuelta para que el navegador pueda pintar la página web.

Aquí tienes el código típico de esta función en Rust explicado paso a paso:

```
// Importa los traits Read y Write (deben ir arriba del todo en el archivo)
use std::io::prelude::*;
use std::net::TcpStream;

fn gestionar_conexion(mut stream: TcpStream) {
    // 1. Creamos el buffer de memoria (la sala de espera temporal)
    // Una matriz de 1024 bytes llenos de ceros
    let mut buffer = [0; 1024];

    // 2. Leemos los bytes que viajan por el stream y los metemos en el buffer
    stream.read(&mut buffer).unwrap();

    // 3. Preparamos la respuesta HTTP correcta (Línea de estado + Cabecera +
    // Cuerpo)
    let contenido = "<html><body><h1>¡Servidor Rust Funciona!</h1></body></html>";
    let respuesta = format!(
        "HTTP/1.1 200 OK\r\nContent-Length: {}\r\n\r\n{}",
        contenido.len(),
        contenido
    );
}
```

```
// 4. Enviamos la respuesta de vuelta por la tubería convirtiendo el texto
en bytes
stream.write_all(respuesta.as_bytes()).unwrap();

// 5. Aseguramos que todos los bytes salgan de la memoria hacia la red
stream.flush().unwrap();
} // <-- Aquí la variable 'stream' se destruye automáticamente y la conexión
// se cierra de forma segura
```

Explicación del paso a paso para la clase:

1. Uso de `unwrap()`: Utilizamos varias veces `unwrap()` en la función porque suponemos que las expresiones que la preceden se van a resolver exitosamente. De nuevo, evitamos de momento el tratamiento de errores pesado para poder razonar sobre un código más limpio y directo.
2. **mut stream: TcpStream**: La variable debe ser mutable (`mut`) porque leer y escribir datos altera el estado interno de la tubería (los bytes entran y salen, modificando los punteros de posición internos).
3. **El Buffer ([0; 1024])**: Creamos una «caja» vacía con capacidad exacta para 1024 bytes. Es el espacio temporal en la memoria RAM donde volcaremos el texto de la petición del navegador.
4. `stream.read(&mut buffer)`: El servidor mete la mano en la tubería, extrae los bytes que envió el navegador y los deposita en nuestro buffer para que podamos inspeccionarlos.
5. **La construcción de la respuesta**: Creamos un String mediante `format!` con la estructura exacta que exige el protocolo HTTP. Usamos los caracteres `\r\n\r\n` (las dos líneas en blanco obligatorias) para separar los metadatos de control del código HTML real que verá el usuario.
6. `stream.write_all(...)`: Transforma nuestro texto a bytes binarios (`.as_bytes()`) y los empuja a través de la tubería de regreso al navegador del usuario.
7. `stream.flush()`: Fuerza al sistema operativo a vaciar los componentes de memoria intermedia interconectados, garantizando que hasta el último byte rezagado sea enviado inmediatamente a la red física.

Al llegar al cierre de la llave `}`, se aplica la magia del Ownership (Propiedad) de Rust: la **variable stream** que fue movida a la función `gestionar_conexion()`, **sale de su ámbito** de existencia (scope), por lo que el compilador inyecta automáticamente la función de liberación de recursos (`drop`) y destruye la variable `stream`. La conexión de red se clausura físicamente en ese instante sin ningún riesgo de fugas de memoria en el servidor.

Al ejecutar este programa y abrir <http://127.0.0.1:8080> en cualquier navegador, verás tu página web. El servidor lee la petición, monta el mensaje HTTP correcto y lo envía de vuelta.

2.6. Servidor web real con control de errores

En el repositorio de GitHub, carpeta **codigo > cuaderno5 > servidor_web_real** hay un proyecto sobre un servidor web real en el que hemos hecho un tratamiento completo de errores. El programa está explicado sobre el mismo listado y pensamos que puede ser interesante

estudiarlo. En esencia es el mismo programa que acabamos de analizar pero quitando los `unwrap()` y realizando un debido tratamiento de errores.

3. Concurrency Segura y Asincronía en Rust

Todos los programas que hemos diseñado en los cuadernos anteriores comparten una característica fundamental: son estrictamente síncronos y no aplican concurrencia. Esto se traduce en que, al ejecutarse, consumen un único hilo de la CPU (procesador).

Sin embargo, la informática actual ha cambiado. La gran mayoría de los procesadores modernos disponen de varios núcleos de hardware, y los sistemas operativos son capaces de gestionar miles de hilos de forma simultánea. Para aprovechar este potencial, los lenguajes modernos incorporan recursos que permiten a los programas exprimir cada núcleo de la CPU.

Para entender los conceptos fundamentales de la concurrencia y la asincronía, haremos un símil entre lo que ocurre realmente en el interior del procesador y una Gran Oficina de Envío de Paquetes (estilo Amazon o Correos) estructurada en tres capas: Hardware, Sistema Operativo y Software (Virtual).

📌 Nota: Con Software (Virtual) nos referimos a hacer que algo parezca real mediante software, aunque físicamente no exista como tal. Por ejemplo, veremos a continuación cómo mediante software podremos crear la ilusión de que estamos ejecutando más tareas simultáneas que núcleos tenemos en la CPU. El software es capaz de conmutar 1.000 tareas entre 8 núcleos y dar la sensación de que se están ejecutando todas a la vez.

3.1. La Analogía de la Oficina de Correos

Imagina que tu programa es una empresa que tiene que procesar millones de paquetes (instrucciones del código).

1. Los Hilos de Hardware (Capa 1) = Las Cintas Transportadoras Físicas

En el sótano de la oficina hay 8 cintas transportadoras físicas instaladas en el suelo. No puedes pulsar un botón mágico y hacer que aparezca una novena cinta de la nada; el edificio se construyó con ocho y punto.

- Esto representa el silicio de tu procesador (los núcleos o hilos de hardware). Es el único lugar donde físicamente se puede procesar una instrucción.

2. Los Hilos del Sistema Operativo (Capa 2) = Los Operarios con Carretilla

Para mover los paquetes hacia las 8 cintas, contratas a Operarios. Cada operario viene con una carretilla enorme, un uniforme pesado, un protocolo estricto y un salario alto fijo (un consumo de memoria de 1 a 8 MB por hilo). Puedes contratar a 100 operarios si quieres, pero recuerda: solo hay 8 cintas físicas disponibles en el hardware.

- **La Conmutación (Context Switch):** Como hay 100 operarios (Hilos del Sistema Operativo) pero solo 8 cintas (Hardware), el Jefe de Planta (el Sistema Operativo) hace sonar un silbato cada pocos milisegundos. Cuando suena, el operario que está en la cinta debe frenar en seco, apuntar exactamente en su libreta por dónde iba, apartar su pesada carretilla y dejar que entre otro operario a la cinta con sus paquetes. Este proceso de intercambio es lento y consume energía de la planta (pérdida de rendimiento en la CPU).

3. Los Hilos de Software o Asincronía (Capa 3) = Los Mensajeros en Patinete

Un buen día, te das cuenta de que la mayoría de tus operarios pesados pierden el tiempo parados en la cinta esperando a que el camión de reparto traiga nuevos paquetes de la calle (esperas de red o lecturas de disco duro). Así que decides cambiar de estrategia: te quedas solo con 8 operarios eficientes (uno fijo en cada cinta de hardware) y, para alimentarlos, contratas a 10.000 Mensajeros en Patinete (Tareas virtuales o Corrutinas). Son ligeros, no ocupan espacio y llevan los paquetes en una mochila pequeña (unos pocos Kilobytes).

- **La magia de la Asincronía:** Si el Mensajero A llega a la cinta y ve que su paquete necesita una firma que tardará en llegar, simplemente da un paso al lado y le dice al Mensajero B: «Oye, pasa tú mientras yo espero». Como son ligeros y no llevan carretilla, cambiar de un mensajero a otro es instantáneo. Los 8 operarios fijos de las cintas nunca paran, y el Jefe de Planta ya no tiene que tocar el silbato. Todo fluye desde dentro del propio programa.

Resumen de Aplicación:

Si tu tarea requiere **fuerza bruta** (cálculo matemático pesado): Necesitas **Operarios con Carretilla** (Concurrencia basada en hilos del SO).

Si tu tarea requiere **gestionar esperas** (consultas web o bases de datos): Necesitas **Mensajeros en Patinete** (Asincronía basada en tareas virtuales o corrutinas).

3.2. Mecanismos de Ejecución en Rust

En el ecosistema de Rust, la concurrencia y la asincronía operan bajo filosofías y costes radicalmente diferentes:

- **Concurrencia (Orientada a la CPU):** El programa levanta hilos nativos del sistema operativo mediante `std::thread::spawn`. Es la herramienta ideal para dividir el trabajo pesado de cálculo. Cada hilo es independiente, síncrono en su interior, y si se queda esperando a que el disco duro lea un archivo, todo ese hilo del sistema operativo se congela por completo en ese instante. Su coste de conmutación es alto y lo gestiona el S.O.
- **Asincronía (Orientada a Entrada/Salida - I/O):** Basada en tareas virtuales (**Futures**). En lugar de bloquear un hilo entero del sistema operativo mientras esperas que internet o la base de datos respondan, el hilo del sistema operativo «suelta» la tarea bloqueada y se dedica a procesar otra inmediatamente, manteniéndose siempre activo al 100% de su eficiencia.

3.3. La conexión entre Concurrencia y Asincronía: Múltiples tareas sobre pocos hilos

La asincronía en Rust es un modelo de concurrencia cooperativa. Te permite ejecutar miles o millones de tareas simultáneas (comportamiento concurrente) utilizando un número mínimo de hilos del sistema operativo (a menudo, tantos hilos como núcleos tenga tu CPU).

Característica	Concurrencia Clásica (<code>std::thread</code>)	Asincronía (<code>async/await</code>)
Enfoque principal	Tareas intensivas de CPU (Cálculo).	Tareas intensivas de I/O (Red, Discos).
Coste de memoria	Alto (Megabytes por hilo del S.O.).	Prácticamente cero (Bytes por tarea).
Gestor	El Sistema Operativo (Planificador).	Un <i>Runtime</i> en espacio de usuario (ej. Tokio).
Bloqueo	Bloquea el hilo por completo.	Pausa la tarea, el hilo sigue libre.

Comentarios sobre la tabla anterior:

1. Coste de memoria: ¿Por qué vemos en la tabla MB por hilo vs. Bytes por tarea? ¿No depende de la tarea a realizar?

Sí depende de la tarea, pero la diferencia radical está en el peaje de entrada que te cobra el sistema operativo por el simple hecho de existir.

- **Concurrencia Clásica (`std::thread`):** Cuando le pides al sistema operativo que cree un hilo nativo, este tiene que reservar obligatoriamente un bloque de memoria fijo llamado Stack (Pila). En la mayoría de sistemas operativos (como Linux o Windows), este tamaño por defecto suele ser de 2 Megabytes. Da igual si tu hilo solo va a sumar $2 + 2$ (que ocupa unos pocos bytes); el sistema operativo ya le ha amputado 2 MB a la memoria RAM de tu ordenador solo para gestionar ese hilo. Si intentas crear 10.000 hilos a la vez, tu servidor colapsará porque necesitará unos 20 GB de RAM solo en «peajes de existencia».
- **Asincronía (`async/await`):** Las «tareas» asíncronas no son hilos del sistema operativo; son gestionadas por Rust como pequeñas estructuras de datos (máquinas de estados). El peaje de entrada aquí es de unos pocos Bytes o Kilobytes (el espacio justo para guardar las variables locales de esa función asíncrona). Si la tarea necesita calcular algo muy grande en el Heap, gastará más, pero si la tarea es pequeña, ocupa casi nada. Por eso un solo hilo de ejecución síncrono (del S. O.) puede gestionar 100.000 tareas web simultáneas en una Raspberry Pi sin despeinarse.

2. ¿Qué significa "en espacio de usuario"?

Un sistema operativo moderno divide la memoria del ordenador en dos zonas de seguridad totalmente aisladas:

- **Espacio de Núcleo (Kernel Space):** Es la zona VIP y ultra-protegida donde vive el corazón del Sistema Operativo. Solo el Kernel puede hablar directamente con la tarjeta de red, el disco duro o el procesador. Crear o destruir un hilo (`std::thread`) requiere que tu programa llame al Kernel, lo cual es lento porque exige un cambio de contexto de seguridad en la CPU.
- **Espacio de Usuario (User Space):** Es la zona donde se ejecutan tus programas normales (como tu navegador, tu juego o tu servidor de Rust).
 - Decir que el Runtime (como Tokio) gestiona la asincronía en «espacio de usuario» significa que es tu propio programa en Rust el que decide qué tarea va primero y cuál va después, mediante código normal y corriente de Rust. No tiene que pedirle permiso al

Kernel del sistema operativo cada vez que cambia de una tarea a otra, lo que hace que cambiar entre miles de tareas asíncronas sea ridículamente rápido y eficiente.

El siguiente capítulo se dedica a la programación concurrente y a continuación tendremos un capítulo dedicado a la programación asíncrona.

4. Concurrency

Como hemos indicado antes, la concurrencia se consigue creando y gestionando hilos del sistema operativo.

Ya hemos comentado las ventajas que ofrece Rust en cuanto a la concurrencia pero, seamos honestos, si programas en Rust de forma síncrona y bloqueante, el programa se va a congelar exactamente igual que en C++, Java o Python. A continuación mostramos un ejemplo de programa bloqueante:

 Copia el siguiente código y ejecútalo en la Playground

Fichero: **descarga_fichero_bloqueante.rs**

```
use std::thread;
use std::time::Duration;

fn descargar_archivo_pesado() {
    println!("[Hilo Principal] Iniciando descarga de 3 segundos...");

    // Esto congela el hilo actual por completo simulando una
    // espera de red o disco
    thread::sleep(Duration::from_secs(3));
    println!("[Hilo Principal] ¡Descarga completada!");
}

fn main() {
    println!("[Hilo Principal] El usuario hace clic en el botón.");

    // Al llamar a la función, el programa se DETIENE aquí durante 3 segundos.
    descargar_archivo_pesado();

    // Esta línea no se ejecutará hasta que la descarga termine.
    // Durante 3 segundos, la interfaz o el juego habrían estado "congelados".
    println!("[Hilo Principal] El usuario ya puede mover el ratón y ver
animaciones.");
}
```

La instrucción `thread::sleep(Duration::from_secs(3));` impone un tiempo de espera de 3 segundos al hilo actual, el hilo `main`, y la función `descargar_archivo_pesado()` simula el tiempo invertido en la descarga de un archivo desde la red o el disco.

Si ejecutas el programa comprobarás que tarda tres segundos en imprimir el mensaje final. Durante esos tres segundos no se han procesado instrucciones del programa. El hilo principal ha estado parado o dormido. El programa no pudo atender al ratón ni al teclado en caso de que lo hubiera necesitado..

Con la concurrencia se puede conseguir que durante esos tres segundos que tarda el hilo principal en descargar el archivo, el procesador esté realizando otra tarea en otro hilo.

En este apartado veremos cómo hacerlo.

4.1. El Motor de la Concurrency: Las Clausuras (Closures)

Antes de ordenar al Sistema Operativo que cree un hilo, debemos entender cómo Rust empaqueta el código que se va a ejecutar en paralelo. Para ello, el lenguaje utiliza una herramienta fundamental llamada **clausura** o **closure**.

¿Qué es una Closure con palabras sencillas? Una closure es una función anónima (sin nombre) que podemos guardar en una variable, pasar como argumento a otras funciones y que, a diferencia de las funciones normales (fn), posee la capacidad de «capturar» y recordar las variables que están escritas en su entorno físico inmediato.

Veamos su evolución progresiva a través de tres ejemplos sencillos:

1. Con parámetros (Como una función): Los parámetros de la closure se colocan entre barras verticales `| |`. El compilador deduce los tipos automáticamente.

```
let sumar = |x, y| x + y;
let resultado = sumar(5, 3); // 8
```

2. Capturando el entorno (Lectura): La closure «atrapa» una variable externa (curso) sin necesidad de recibirla por parámetro.

```
let curso = "Rust";
let info = || println!("Curso: {}", curso);
info(); // Curso: Rust
```

3. Modificando el entorno (Paso previo a los hilos): Si la closure quiere modificar lo que hay fuera, debemos marcarla como mutable (`mut`).

```
let mut cuenta = 0;
let mut incrementar = || {
    cuenta += 1;
    println!("Cuenta: {}", cuenta);
};
incrementar(); // Cuenta: 1
```

ⓘ Nota: Si intentáramos replicar los ejemplos 2 y 3 usando funciones tradicionales con 'fn', el compilador arrojará un error inmediato. Las funciones normales no tienen memoria de su entorno; las closures sí.

4.2. El problema de los hilos y la solución con `move`

Los hilos se crean usando `thread::spawn` y enviándole una closure.

Cuando intentamos enviar una closure a otro hilo usando `thread::spawn`, el compilador de Rust interviene para protegernos de un peligro crítico de memoria:

```
use std::thread;
```

```
fn main() {
    let saludo = String::from("¡Hola desde el hilo!");

    // ✗ --- error: El hilo podría vivir más tiempo que la variable 'saludo'
    thread::spawn(|| {
        println!("{}", saludo);
    });
}
```

¿Por qué falla?

Por defecto, la closure intenta tomar «prestada» (mediante una referencia) la variable `saludo`. Sin embargo, como el hilo secundario se ejecuta en paralelo de forma independiente, el hilo principal (`main`) podría terminar antes de tiempo, limpiar la memoria y destruir la variable `saludo`. Si eso ocurriera, el hilo secundario intentaría leer un espacio de memoria vacío o corrupto.

El trabajo de la palabra clave `move`

Para solucionar este dilema, anteponeamos la palabra clave `move` antes de las barras de la closure.

```
// SOLUCIÓN COMPILADA CON ÉXITO
let handle = thread::spawn(move || {
    println!("{}", saludo); // La variable ahora vive aquí dentro
});
```

Al añadir `move`, el compilador altera drásticamente el comportamiento de la memoria:

1. **Transferencia de propiedad (Ownership):** La closure deja de pedir prestado. Corta los lazos con el hilo principal y se adueña físicamente de la variable `saludo`, mudándola al espacio de memoria del nuevo hilo.
2. **Aislamiento seguro:** Como la variable ahora le pertenece por completo al hilo secundario, da igual si `main` termina antes o después; los datos están a salvo del borrado accidental.
3. **Restricción en el origen:** Como consecuencia de la mudanza, el hilo principal pierde todos los derechos sobre la variable. Intentar usar `saludo` en `main` después de un `move` causará un error de compilación.

En una sola frase: El trabajo de `move` es transformar una closure que **toma prestados** datos del entorno en una closure que **se adueña** de ellos, logrando que el nuevo hilo sea 100% autónomo y seguro.

4.3. Las formas de crear hilos

Para solucionar el problema del programa bloqueante que vimos al principio, abrimos «líneas temporales paralelas» (Hilos de software). En Rust básico hay dos formas principales de gestionar esto mediante la biblioteca estándar.

- Forma A: El hilo independiente (**`thread::spawn con clausura`**)
 - Se corresponde con los ejemplos anteriores utilizados en la explicación de las closures.

- Forma B: Hilos con Ámbito o Concurrency Estructurada (**thread::scope**)

4.3.1. A: El hilo independiente (**thread::spawn con clausura**)

Lanzamos un hilo y nos olvidamos de él temporalmente mientras hacemos otra cosa en el hilo principal.

 Copia el siguiente código y ejecútalo en la Playground

Fichero: **hilo_independiente.rs**

```
use std::thread;
use std::time::Duration;

fn main() {
    println!("[Hilo Principal] Iniciando el programa.");

    // FORMA A: Creamos un hilo secundario independiente
    let manejador = thread::spawn(|| {
        // ---- TODO ESTE BLOQUE SE EJECUTA EN EL HILO SECUNDARIO ----
        println!("[Hilo Secundario] ¡Hola! Estoy descargando el archivo en paralelo...");
        thread::sleep(Duration::from_secs(3));
        println!("[Hilo Secundario] Descarga finalizada en segundo plano.");
        // -----
    });

    // ---- TODO ESTO SIGUE EJECUTÁNDOSE EN EL HILO PRINCIPAL EN PARALELO ----
    for i in 1..4 {
        println!("[Hilo Principal] Yo sigo libre. Dibujando fotograma {}",
i);
        thread::sleep(Duration::from_millis(500));
    }

    println!("[Hilo Principal] Ya no tengo más que hacer, ahora sí espero al secundario.");
    // Obligamos al hilo principal a esperar a que el secundario termine si no lo ha hecho
    manejador.join().unwrap();

    println!("[Hilo Principal] Fin del programa.");
}
```

❗ Nota: La instrucción **thread::sleep(Duration::from_secs(3));** produce una espera en el hilo que se ejecute de 3 segundos. En este caso en el hilo secundario.

La ejecución del programa debe producir la siguiente salida:

```
[Hilo Principal] Iniciando el programa.
[Hilo Principal] Yo sigo libre. Dibujando fotograma 1...
[Hilo Secundario] ¡Hola! Estoy descargando el archivo en paralelo...
[Hilo Principal] Yo sigo libre. Dibujando fotograma 2...
[Hilo Principal] Yo sigo libre. Dibujando fotograma 3...
```

```
[Hilo Principal] Ya no tengo más que hacer, ahora sí espero al secundario.
[Hilo Secundario] Descarga finalizada en segundo plano.
[Hilo Principal] Fin del programa.
```

- Con **let manejador = thread::spawn(|| {...** creamos el hilo y obtenemos el **manejador** para obligar al hilo principal (main) a esperar a que termine el hilo secundario si no lo ha hecho ya.
- La instrucción **manejador.join().unwrap();** es la que hace esperar al hilo principal. El **.unwrap()** desempaqueta de forma obligatoria lo que tenemos a la izquierda. En este caso lo suponemos seguro aunque bajo ciertas condiciones muy poco probables podría fallar. Evitamos el tratamiento de errores para simplificar el código.

4.3.2. B: Hilos con Ámbito o Concurrencia Estructurada (thread::scope)

Esta forma se añadió en versiones más recientes de Rust. Te permite crear hilos que garantizan que terminarán antes de que acabe el bloque de código (ver listado), lo que te permite compartir variables locales del hilo principal sin usar trucos raros.

La closure toma «prestada» (mediante una referencia) la variable del main que va a utilizar. Si el hilo principal (main) pudiera terminar antes que el secundario, podría limpiar la memoria y destruir la variable como ya hemos mencionado antes. Si eso ocurriera, el hilo secundario intentaría leer un espacio de memoria vacío o corrupto. Con **thread::scope** se garantiza que los hilos secundarios creados terminarán antes que el main.

 Copia el siguiente código y ejecútalo en la Playground

Fichero: **hilo_scope.rs**

```
use std::thread;
use std::time::Duration;

fn main() {
    let mut datos_compartidos = vec![1, 2, 3];

    println!("[Hilo Principal] Iniciando ámbito de hilos.");

    // FORMA B: Creamos un "ámbito" seguro
    thread::scope(|scope| {
        // Lanzamos el Hilo Secundario 1 dentro del scope
        scope.spawn(|| {
            println!("[Hilo Secundario 1] Leyendo datos: {:?}",
datos_compartidos);
            thread::sleep(Duration::from_millis(200));
        });

        // Lanzamos el Hilo Secundario 2 dentro del mismo scope
        scope.spawn(|| {
            println!("[Hilo Secundario 2] Yo también puedo verlos: {:?}",
datos_compartidos);
        });

        // ---- EL HILO PRINCIPAL TAMBIÉN PUEDE TRABAJAR AQUÍ ----
    });
}
```

```

        println!("[Hilo Principal] Trabajando dentro del scope...");
    });
    // <--- Al llegar aquí, el 'scope' se bloquea automáticamente y ESPERA
    // a que todos los hilos secundarios terminen. No hace falta usar .join() a
    mano.

    println!("[Hilo Principal] Fuera del scope. Todos los hilos han muerto con
    certeza.");
}

```

datos_compartidos permanece en todo momento propiedad del hilo principal main.

La salida del programa debe ser la siguiente:

```

[Hilo Principal] Iniciando ámbito de hilos.
[Hilo Principal] Trabajando dentro del scope...
[Hilo Secundario 2] Yo también puedo verlos: [1, 2, 3]
[Hilo Secundario 1] Leyendo datos: [1, 2, 3]
[Hilo Principal] Fuera del scope. Todos los hilos han muerto con certeza.

```

El scope (ámbito) se crea co la sintaxis: **thread::scope(|scope| {...**

Observaciones

¿Por qué se puede acceder desde dos hilos distintos a la misma variable? Porque es solo lectura y porque el scope garantiza que esos hilos no sobrevivirán a la variable **datos_compartidos** del main.

Recordemos la: 💡 Regla de oro de Rust: Puedes tener infinitas referencias de solo lectura (&T) a un dato al mismo tiempo, O puedes tener una única referencia mutable (&mut T), pero nunca ambas cosas a la vez.

En el código, cuando hacemos **println!("{:?}", datos_compartidos)** dentro de los hilos, el compilador detecta que los hilos solo necesitan leer el vector. Por lo tanto, el scope les permite hacer un préstamo de solo lectura (&datos_compartidos).

Y si modificamos la variable, podríamos hacerlo desde los dos hilos?

Depende. Si intentas modificarla alegremente como en otros lenguajes, el compilador de Rust detendrá el programa en seco antes de compilarlo.

Veamos los dos escenarios posibles de mutación:

A: Un hilo intenta modificarla mientras otro la lee (ERROR)

Si intentas hacer esto:

```

thread::scope(|scope| {
    // Hilo 1 intenta MODIFICAR
    scope.spawn(|| {
        datos_compartidos.push(4); // ❌ Intenta pedir un préstamo mutable
    (&mut)
    });
});

```

```
// Hilo 2 intenta LEER
scope.spawn(|| {
    println!("{:?}", datos_compartidos); // ❌ Intenta pedir un préstamo de
    lectura (&)
});
});
```

El compilador no te dejará. Te dará un error diciendo que no puedes prestar **datos_compartidos** como mutable e immutable al mismo tiempo. ¿El motivo físico? Si el Hilo 1 añade un elemento, el vector podría necesitar reasignar su memoria RAM justo en el microsegundo en que el Hilo 2 intenta leerlo, provocando que el Hilo 2 apunte a memoria basura (un bug gravísimo).

B: Un ÚNICO hilo la modifica dentro del scope (PERMITIDO)

Si solo un hilo la usa y la modifica, sí se puede:

```
thread::scope(|scope| {
    // Un único hilo se adueña del acceso mutable
    scope.spawn(|| {
        datos_compartidos.push(4); // ¡Permitido! Nadie más la está mirando
    });
});
```

4.4. Compartir Datos Mutables: El Poder del Mutex

Hasta ahora hemos visto cómo move aísla los hilos transfiriendo la propiedad de las variables. Pero en el mundo real, a menudo necesitamos que **varios hilos accedan y modifiquen los mismos datos en paralelo**.

Si dos hilos intentan escribir en la misma porción de memoria al mismo tiempo, se produce una **condición de carrera** (race condition), lo que corrompe los datos o rompe el programa. Rust prohíbe esto en tiempo de compilación. Para lograrlo de forma segura, necesitamos dos herramientas que trabajan en equipo: el **Mutex** y el **Arc**.

Qué es el struct `Arc<Mutex<T>>`

Veamos:

1. **T** es simplemente una caja genérica. Representa cualquier tipo de dato que queramos proteger (un número, un vector, una estructura).
2. **Mutex** actúa sobre esa T para ponerle el cerrojo.
3. **Arc** envuelve al Mutex entero para permitir que varios hilos sean «dueños» de esa misma caja al mismo tiempo.
4. ¡Arc podría envolver a otras cosas que no sean un Mutex!

Vamos a utilizar la siguiente analogía: **La Caja Fuerte Portátil**

Imagínate que tenemos un tesoro (nuestra T, que en el ejemplo de abajo será un simple número contador).

- **Mutex<T> (El Cerrojo):** Metemos nuestro contador dentro de una caja fuerte. La caja tiene una única llave adjunta. Si un hilo quiere ver o cambiar el contador, tiene que pedir la llave (`.lock()`). Mientras ese hilo tenga la llave, nadie más puede abrir la caja.
- **Arc<Mutex<T>> (El Duplicador de Propiedad):** Aquí viene el problema de Rust. Por las reglas de ownership, un objeto solo puede tener un único dueño. Si le entregamos la caja fuerte al Hilo 1, el Hilo 2 ya no puede tenerla. Para solucionar esto, metemos la caja fuerte dentro de un envoltorio **Arc** (significa Atomic Reference Counted). El Arc nos permite hacer clones exactos del contenedor. Ojo: no clona lo que hay dentro de la caja fuerte, sino que crea «mandos a distancia» o accesos compartidos a la misma caja fuerte física.

Por tanto, **Arc<Mutex<T>** se traduce como: Un acceso compartido (**Arc**) a una caja fuerte (Mutex) que protege a un dato genérico (**T**).

En el siguiente programa, pondremos a 3 hilos independientes a incrementar el mismo número **contador** a la vez. Verás lo honesto y robusto que es el código:

 Copia el siguiente código y ejecútalo en la Playground

Fichero: **hilos_mutex.rs**

```
use std::sync::{Arc, Mutex};
use std::thread;
use std::time::Duration;

fn main() {
    // 1. Creamos el dato original (un número 0) protegido por el Mutex
    // y compartido por el Arc
    // T en este caso resulta ser un entero 'i32'
    let contador_compartido = Arc::new(Mutex::new(0));

    // Guardaremos los manejadores de los hilos para sincronizarlos al final
    let mut manejadores = vec![];

    println!("[Hilo Principal] Lanzando 3 hilos obreros...");

    for id_hilo in 1..=3 {
        // 2. ¡EL PASO CLAVE! Clonamos el 'Arc'.
        // Esto NO duplica el número 0 de la memoria. Crea un "acceso
        // numerado" nuevo hacia la misma caja fuerte original.
        let contador_clon = Arc::clone(&contador_compartido);

        let manejador = thread::spawn(move || {
            // ---- CÓDIGO DENTRO DEL HILO SECUNDARIO ----
            println!("[Hilo {}] Esperando mi turno para abrir la caja...",
id_hilo);

            // 3. Abrimos el cerrojo. Si otro hilo lo está usando,
            // este hilo se detiene a esperar.
            // '.unwrap()' se usa por si la caja se rompe (pánico en otro
            hilo).
            let mut dato_interno = contador_clon.lock().unwrap(); // lock()
            devuelve Result

            // 4. Modificamos el valor con total seguridad
```

```

        *dato_interno += 1;

        println!("[Hilo {}] He incrementado el contador a: {}", id_hilo,
*dato_interno);

        // Simulamos que el hilo tarda un poco haciendo cosas con la caja
abierta
        thread::sleep(Duration::from_millis(50));

        // 5. ¡Magia automática de Rust!
        // Al llegar al final de la función del hilo, la variable
'dato_interno' se destruye
        // y el Mutex SE CIERRA SOLO de forma automática liberando la llave
para el siguiente hilo.
    });

    manejadores.push(manejador);
}

// 6. El hilo principal espera a que los 3 terminen
for manejador in manejadores {
    manejador.join().unwrap();
}

// 7. El hilo principal pide la llave por última vez para ver el resultado
final
let resultado_final = contador_compartido.lock().unwrap();
println!("[Hilo Principal] Todos terminaron. El valor final es: {}",
*resultado_final);
}

```

Aclaraciones

En el programa hay dos tipos de `unwrap()` que hacen trabajos de seguridad muy específicos:

1. El `unwrap()` después de `.lock()`

Cuando hacemos `contador_compartido.lock()`, Rust no te devuelve directamente el número. Te devuelve un `Result`. ¿Por qué puede fallar abrir un cerrojo?

- Si el Hilo 1 abre la caja fuerte (`.lock()`) y mientras tiene el dinero en la mano sufre un error grave imprevisto (un pánico, por ejemplo, si dividieras por cero ahí dentro), el hilo muere de golpe.
- Al morir así, el cerrojo se queda en un estado especial llamado envenenado (`poisoned`).
- Si el Hilo 2 intenta hacer `.lock()` en una caja envenenada, Rust te avisa devolviendo un `Err` para decirte: «Oye, el último que entró aquí murió de forma violenta y los datos dentro de la caja podrían estar corruptos».
- Al poner `.unwrap()`, estás diciendo: «Si la caja está envenenada, prefiero que mi hilo también muera en ese mismo instante».

2. El `unwrap()` después de `.join()`

Cuando haces `manejador.join()`, le dices al hilo principal que espere a que el hilo secundario termine. `.join()` también devuelve un `Result`.

- Si el hilo secundario termina su trabajo felizmente, `.join()` devuelve `Ok`. El `.unwrap()` lo lee, ve que todo está bien y el programa continúa.
- Si el hilo secundario hubiera muerto por un pánico interno, `.join()` devolvería un `Err`. El `.unwrap()` capturaría ese error y haría que el hilo principal también explotara, evitando que el juego continúe con datos corruptos o hilos zombies.

¿Por qué esta combinación es tan sincera y potente en Rust?

Otros lenguajes también usan `Mutex`. La diferencia crucial aquí es la línea: `*dato_interno += 1`.

El asterisco `*` se llama operador de desreferenciación. Rust no te deja modificar la variable **contador_clon** directamente. Te obliga a pasar por el peaje del `.lock()`. Al obtener el cerrojo, te devuelve un «puntero inteligente» (`MutexGuard`), y solo al desreferenciarlo con `*` puedes tocar el número de verdad.

Si olvidas poner el `.lock()`, el compilador de Rust detendrá la compilación gritando: «¡Error! No puedes modificar un `Arc` directamente, necesitas abrir el `Mutex` primero!». Eso es Concurrency sin miedo.

4.5. Los Canales MPSC: pasar Mensajes en lugar de compartir Memoria

Hasta ahora hemos aprendido a coordinar hilos mediante **memoria compartida**: metemos los datos dentro de un cofre (`Mutex`) y hacemos que los hilos se peleen por el cerrojo. Aunque es un sistema robusto, puede volverse complejo y provocar atascos si muchos hilos compiten a la vez.

Existe una filosofía alternativa y sumamente elegante: **la transferencia de mensajes**. En lugar de que los hilos compartan un cofre de datos, los hilos trabajan de forma aislada y se envían los resultados a través de un **Canal**.

Para muchos programadores la utilización de canales MPSC es la forma más elegante porque evita tener que lidiar con cerrojos, llaves o `Arc<Mutex<T>>`.

¿Qué significa MPSC?

Las siglas MPSC significan Multi-Producer, Single-Consumer (Múltiples Productores, Un solo Consumidor). La analogía perfecta es un Río donde confluyen varios arroyos, o un Buzón de correos de una oficina:

- Varios hilos obreros (los «Productores») pueden escribir mensajes y lanzarlos por el canal.
- Pero solo un hilo central (el «Consumidor», que suele ser el Hilo Principal) está sentado al final del canal escuchando y procesando los mensajes que van llegando, uno a uno y en orden.

A diferencia del `Mutex` (donde los hilos se pelean por la misma variable), aquí los hilos secundarios simplemente envían copias de sus datos a través de una tubería hacia el hilo principal.



Copia el siguiente código y ejecútalo en la Playground

Fichero: **hilos_canales.rs**

```
use std::sync::mpsc; // Importamos el módulo de canales
use std::thread;
```

```

use std::time::Duration;

fn main() {
    // 1. Creamos el canal. Nos devuelve una tupla con:
    // tx: El Transmisor (Transmitter) -> Para enviar datos
    // rx: El Receptor (Receiver)      -> Para recoger datos
    let (tx, rx) = mpsc::channel();

    println!("[Hilo Principal] Creando hilos obreros...");

    for id_obrero in 1..=3 {
        // 2. Como MPSC permite MÚLTIPLES productores, clonamos el
        // transmisor 'tx' para darle una copia a cada hilo nuevo que creamos.
        let tx_clonado = tx.clone();

        thread::spawn(move || {
            // ---- CÓDIGO DENTRO DEL HILO OBRERO ----

            // Esperas escalonadas
            thread::sleep(Duration::from_millis(id_obrero * 100));

            let mensaje = format!("Hola desde el obrero {}", id_obrero);

            // 3. El hilo lanza el mensaje por su copia de la tubería
            tx_clonado.send(mensaje).unwrap();

            // Aquí el hilo termina y muere de forma limpia.
        });
    }

    // 4. ¡TRUCO CRUCIAL DE RUST!
    // El 'tx' original que creamos en la línea 8 sigue vivo
    // en el hilo principal.

    // Si no lo destruimos o dejamos caer, el Receptor se quedará
    // esperando eternamente pensando que el hilo principal aún podría
    // enviar algo. Al soltarlo, el canal sabe que solo quedan vivos los
    // transmisores clonados de los hilos obreros.
    drop(tx);

    println!("[Hilo Principal] Sentado a esperar mensajes en el receptor...");

    // 5. El hilo principal se queda leyendo el receptor 'rx' en un bucle.
    // Este bucle 'for' se bloquea pacientemente esperando paquetes.
    // Cuando todos los hilos obreros mueren y sus transmisiones se cierran,
    // el bucle termina solo.
    for mensaje_recibido in rx {
        println!("[Hilo Principal] He recibido: '{}'", mensaje_recibido);
    }

    println!("[Hilo Principal] Canal cerrado. Todas las tareas terminaron.");
}

```

La salida del programa debe ser la siguiente:

```
[Hilo Principal] Creando hilos obreros...
[Hilo Principal] Sentado a esperar mensajes en el receptor...
[Hilo Principal] He recibido: 'Hola desde el obrero 1'
[Hilo Principal] He recibido: 'Hola desde el obrero 2'
[Hilo Principal] He recibido: 'Hola desde el obrero 3'
[Hilo Principal] Canal cerrado. Todas las tareas terminaron.
```

La Magia de la Propiedad en los Canales

Presta mucha atención a lo que ocurre en la línea `tx_clonado.send(mensaje):`

- En otros lenguajes, enviar un objeto por un canal puede provocar que dos hilos lean el mismo objeto a la vez, corrompiendo la memoria.
- En Rust, la función `.send()` **toma la propiedad (ownership)** de la variable. En cuanto el hilo secundario envía el `String`, ese hilo ya no puede volver a usarlo. El dato viaja flotando por el canal de manera 100% segura hasta que `main` lo recibe y se adueña de él. ¡No hay posibilidad de conflictos en la memoria!

Resumen para comparar: Mutex o Canales?

La regla de oro es la siguiente:

- Si los hilos necesitan modificar la misma estructura a la vez (ej. una lista de usuarios conectados en un servidor de juego): Usa `Arc<Mutex<T>>`.
- Si los hilos solo hacen un trabajo independiente en segundo plano y te quieren mandar el resultado final cuando terminen: Usa Canales `mpsc`.

5. Asincronia: `async` / `await` en Rust

En Rust, el modelo asíncrono permite ejecutar tareas sin bloquear el hilo principal de ejecución, logrando una eficiencia masiva.

El fundamento es sencillo. Cuando una tarea que declaradmos asíncrona, por ejemplo la descarga de un fichero o la entrada de datos por parte del usuario, tiene que esperar bien a que termine la descarga o a que el usuario termine con la entrada de datos, se pausa la tarea y el hilo sigue trabajando en su tarea principal o en otra tarea asíncrona que esté lista. Esta comutación entre tareas es muy rápida y con muy poco costo de memoria para el sistema comparada con la conmutación entre hilos del sistema operativo.

Rust proporciona la sintaxis y las herramientas básicas pero para programar la asincronía necesitas una librería externa o `runtime` como **tokio** o **async-std**.

Qué es un futuro?

Un Future (futuro) es un valor que representa una operación que aún no ha terminado, pero que promete completarse en algún momento.

Cómo se utiliza `async` / `await`

La sintaxis `async` y `await` son palabras clave que transforman bloques de código normales en estructuras que devuelven futuros y permiten pausar la ejecución sin bloquear el hilo que las ejecuta.

- **`async`**: Transforma un bloque de código o una función para que devuelva un objeto que implementa Future.
- **`await`**: Pausa la ejecución de la función actual marcada con `async` hasta que el futuro esté listo, permitiendo que el hilo ejecute otras tareas mientras tanto.

5.1. Ejemplo práctico con **tokio**

- Abre una terminal integrada de VS Code en la carpeta **proyectos-rust** (o en cualquier otra) y crea el proyecto de Rust **tokio_ejemplo_asin** con la orden:

```
cargo new tokio_ejemplo_asin
```

Edita la sección [dependencies] del fichero Cargo.toml como se muestra a continuación:

```
[dependencies]
tokio = { version = "1", features = ["full"] }
```

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado, borrando previamente todo lo que hubiera escrito en ese fichero.

Proyecto: **tokio_ejemplo_asin**

src/main.rs

```

use std::time::Duration;
use tokio::time::sleep;

// 1. Definimos una función asíncrona usando 'async'
async fn descargar_archivo(id: u32) -> String {
    println!("Iniciando descarga del archivo {}", id);

    // Simulamos una espera de red de 2 segundos sin bloquear el hilo
    sleep(Duration::from_secs(2)).await;

    format!("Contenido del archivo {}", id)
}

// 2. La función principal también debe ser asíncrona y gestionada
// por un runtime. En este caso hemos elegido 'tokio'
#[tokio::main]
async fn main() {
    println!("Iniciando el gestor de descargas.");

    // Creamos dos futuros (no se ejecutan todavía)
    let descarga1 = descargar_archivo(1);
    let descarga2 = descargar_archivo(2);

    // Con tokio::join! ejecutamos ambos futuros en paralelo de forma asíncrona
    // El programa tardará ~2 segundos en total, no 4.
    let (resultado1, resultado2) = tokio::join!(descarga1, descarga2);

    println!("Resultado 1: {}", resultado1);
    println!("Resultado 2: {}", resultado2);
    println!("¡Todas las descargas finalizadas!");
}

```

La salida del programa debe ser la siguiente:

```

Iniciando el gestor de descargas.
Iniciando descarga del archivo 1...
Iniciando descarga del archivo 2...
Resultado 1: Contenido del archivo 1
Resultado 2: Contenido del archivo 2
¡Todas las descargas finalizadas!

```

5.2. Ventajas frente a la concurrencia con Hilos del Sistema Operativo (OS Threads)

Rust permite usar hilos nativos mediante `std::thread`, pero el modelo `async/await` ofrece ventajas críticas en escenarios de alta carga (como servidores web y aplicaciones de red o de E/S):

Característica	Hilos del Sistema Operativo (<code>std::thread</code>)	Asincronía (<code>async/await</code>)
----------------	---	---

Modelo	Concurrencia preventiva (el OS gestiona el tiempo).	Concurrencia cooperativa (las tareas ceden el control).
Consumo de Memoria	Alto. Cada hilo reserva un gran bloque de memoria para su propia pila (<i>stack</i>), usualmente de 1 a 2 MB.	Muy Bajo. Las tareas asíncronas no tienen pilas propias; se guardan en el <i>heap</i> ocupando solo unos pocos bytes o kilobytes.
Cambio de Contexto	Costoso. Cambiar de un hilo a otro requiere que la CPU intervenga y modifique registros del sistema.	Muy rápido. El cambio ocurre dentro del propio <i>runtime</i> en el espacio de usuario, casi gratis a nivel de CPU.
Escalabilidad	Limitada a unos pocos miles de hilos antes de que el sistema se sature.	Permite manejar millones de tareas simultáneas en una sola máquina.

En resumen: ¿Cuándo usar cada uno?

- Usa Hilos del OS si tus tareas requieren un uso intensivo de la CPU (matemáticas pesadas, procesamiento de imágenes o criptografía).
- Usa Asincronía si tus tareas pasan la mayor parte del tiempo esperando (E/S), como servidores web, peticiones API, bases de datos o lecturas de archivos masivos.

5.3. El Rol del Runtime (Ej. Tokio o el de Macroquad)

Dado que la biblioteca estándar de Rust no incluye un planificador asíncrono para mantener los binarios lo más pequeños posibles, necesitas un Runtime externo (como Tokio para servidores o el interno de Macroquad para juegos como veremos luego).

Que funciones desempeña el runtime?

- El Runtime mantiene una cola de Futures.
- Llama a `poll()` en el Future para preguntar su estado.
- Si el Future devuelve `Poll::Pending`, el Runtime lo registra en el sistema operativo (usando herramientas eficientes como `epoll` en Linux o `IOCP` en Windows) para que el hardware le avise cuando haya actividad en la tarjeta de red o el disco.
- Mientras tanto, ese hilo del Runtime pasa a procesar otro Future de la cola.
- Cuando el S.O. avisa que los datos llegaron, el Runtime despierta al Future correspondiente mediante un componente llamado `Waker` y vuelve a llamar a `poll()`, avanzando al siguiente estado.

5.4. ¿Por qué llamamos a Macroquad Runtime interno?

Llamamos a **Macroquad** «runtime interno» porque el propio motor del juego incluye e implementa su propio **planificador asíncrono** dentro de su código base, sin obligarte a depender de herramientas externas como **Tokio** o **async-std**.

En el siguiente apartado utilizaremos Macroquad para programar un juego.

En Rust, la biblioteca estándar provee la sintaxis (`async`, `await`) y el `trait` base (`Future`), pero no incluye el motor que hace rodar esos futuros. Alguien tiene que encargarse de llamar a `poll()` y gestionar la cola de tareas.

Macroquad necesita controlar de forma milimétrica el tiempo y los fotogramas del juego. Por ello, en lugar de usar un runtime de propósito general (como Tokio, que está diseñado para servidores web y bases de datos), implementa su propio **bucle de control asíncrono** optimizado para gráficos:

1. Gestión del bucle gráfico: El macro-bucle de Macroquad toma tu función `async fn main()` y ejecuta su código frame a frame.
2. Control del `.await`: Cuando tu código llega a `next_frame().await`, el planificador interno de Macroquad sabe exactamente que debe pausar tu función `main`, cederle el control a la tarjeta gráfica para que dibuje la pantalla y, justo en el siguiente fotograma, volver a despertar tu función para continuar con la lógica.

En resumen, se le llama «interno» porque viene dentro de la propia caja de Macroquad, permitiendo que los juegos en Rust funcionen de forma asíncrona con una configuración cero para el programador.

En el siguiente apartado comenzamos a trabajar con Macroquad así que esto ha sido un avance para poderlo comparar con la opción `tokio` utilizada en el ejemplo anterior.

6. Proyecto Final de Ciclo - Videojuego Gráfico Macroquad

"¡Ajá!" vamos a ver cómo la teoría se convierte en un videojuego interactivo que responde en tiempo real.

Para este proyecto utilizaremos Macroquad, una librería de Rust increíblemente ligera que no requiere configuraciones complejas de sistemas operativos ni herramientas pesadas, ideal para los ordenadores del instituto.

Para conectar de forma brillante el mundo de los hilos con el de los videojuegos usando Macroquad, debemos estructurar la explicación en torno a una sola pregunta: ¿Por qué no usamos los hilos independientes (`std::thread`) que acabamos de aprender para mover los personajes del juego?

1. El Puente Pedagógico: El problema de los hilos en los videojuegos

Si intentáramos programar un videojuego creando un hilo para el jugador, otro hilo para la pelota y otro hilo para los enemigos, nos topáramos con dos problemas catastróficos:

- **El Caos del Sincronismo:** Los hilos van a su propio ritmo. Si el hilo de la pelota calcula su posición a mitad de camino mientras el hilo del jugador se está moviendo, la pelota podría atravesar la pala antes de que el procesador dibuje la imagen. El juego daría tirones horribles.
- **Coste Innecesario:** Como vimos en la tabla, crear un hilo cuesta 2 MB de memoria. Un juego con 50 enemigos gastaría 100 MB solo en «peajes de hilos» para mover simples numeritos en la pantalla

La Solución de Macroquad: Usar Asincronía Cooperativa (`async/await`) sobre un solo hilo principal.

2. Cómo funciona la magia de Macroquad por dentro

Cuando abras el proyecto del juego, te encontrarás con esta cabecera obligatoria:

```
#[macroquad::main("Mi Juego")]
async fn main() {
    loop {
        // ... Lógica del juego ...
        next_frame().await; // <--- El punto de conexión clave
    }
}
```

¿Qué está pasando realmente aquí?

- Un único hilo de la CPU ejecuta todo el juego. No hay peligro de corrupción de memoria ni peleas por variables. No hacen falta `Arc<Mutex>` porque una sola línea temporal lee los datos en orden de arriba a abajo.

- La palabra clave **async** le avisa a Rust de que esta función `main` es una «máquina de estados» que tiene permiso para pausarse en medio de su ejecución.
- La palabra clave **.await** es el botón de pausa cooperativo.

3. La analogía definitiva: El Director de Cine y el Dibujante

Para entenderlo de forma visual:

- El bucle `loop` es un Director de Cine. Calcula dónde debe estar el jugador y la pelota en el siguiente fotograma.
- Al llegar a **`next_frame().await`**, el Director se para en seco, levanta la mano y le dice al hilo del motor gráfico: «Oye, ya he calculado las posiciones. Te cedo el control. Dibuja el fotograma en la pantalla y, cuando la tarjeta gráfica termine, avísame para que continúe».
- Mientras la tarjeta gráfica trabaja, el hilo de software no se congela de forma destructiva; simplemente cede el control a la espera de la señal gráfica. Cuando el fotograma se muestra en el monitor, la máquina de estados se «despierta» en el mismo punto exacto y arranca la siguiente iteración del bucle.

4. El Gran Cierre: ¿Y si metemos canales `mpsc` en el juego?

Para demostrar que todo lo aprendido en el cuaderno está conectado, podemos plantear el desafío de crear analíticas o logs que mencionamos al principio.

Imagina que el juego tiene que guardar en un archivo de texto cada vez que el jugador mete un punto. Escribir en el disco duro es una operación síncrona lenta que congelaría la pantalla del juego un milisegundo (provocando un molesto tirón o lag).

¿Cómo lo solucionamos uniendo los dos mundos?

- Creamos un canal `mpsc` antes de arrancar el juego.
- Lanzamos un hilo secundario independiente (`std::thread::spawn`) fuera del juego que se queda escuchando el receptor (`rx`). Su único trabajo es escribir en el disco duro.
- El bucle asíncrono del juego (`Macroquad`) se queda con el transmisor (`tx`). Cada vez que hay un punto, lanza un mensaje por el canal y sigue jugando a 60 FPS sin detenerse ni un microsegundo. El hilo secundario se encargará del trabajo sucio de fondo.

6.1. Proyecto Guiado: Nuestro primer juego gráfico (Clon de Pong)

Aquí tienes la guía paso a paso para el desarrollo del clon de Pong, diseñada específicamente para el Cuaderno 5. Esta guía conecta de manera directa la lógica del juego con el funcionamiento de la máquina de estados asíncrona de `Macroquad`.

En este proyecto práctico vamos a construir el clásico juego de Pong. La clave de este desarrollo es entender cómo el Game Loop (Bucle de juego) gestiona todo de forma asíncrona en un solo hilo, pausándose exactamente cuando se lo pedimos para sincronizarse con la tarjeta gráfica.

Paso 1: Configurar el entorno de juego (`Cargo.toml`)

 Abre una terminal integrada de VS Code en el directorio **proyectos-rust** creado al principio o en cualquier otro y crea un proyecto Rust con la orden: **`cargo new juego_pong`**.

Sitúate en la carpeta del proyecto mediante **cd juego_pong** e instala la dependencia **macroquad** con la orden: **cargo add macroquad**.

Tu fichero Cargo.toml debe lucir algo así:

```
[package]
name = "juego_pong"
version = "0.1.0"
edition = "2024"

[dependencies]
macroquad = "0.4.16"
```

Paso 2: Definir las Entidades del juego

En lugar de tener variables sueltas por el código, utilizaremos el sistema de estructuras (struct) de Rust para modelar los dos elementos clave de la pantalla: las palas (rectángulos) y la pelota (un círculo).

 Copia el siguiente código en el archivo src/main.rs

src/main.rs

```
use macroquad::prelude::*;

struct Pala {
    x: f32,
    y: f32,
    ancho: f32,
    alto: f32,
    velocidad: f32,
}

struct Pelota {
    x: f32,
    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}
```

Paso 3: Inicializar la máquina de estados (async fn main)

El punto de entrada del juego no es una función normal. Lleva el macro **#[macroquad::main]** y está marcada como **async**. Esto le dice al compilador de Rust que prepare la infraestructura asíncrona interna del motor para controlar el tiempo.

Tu programa debe tener este aspecto:

```
use macroquad::prelude::*;

struct Pala {
    x: f32,
    y: f32,
```

```

    ancho: f32,
    alto: f32,
    velocidad: f32,
}

struct Pelota {
    x: f32,
    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}

#[macroquad::main("Rust Pong Educativo")]
async fn main() {
    // Inicializamos al jugador en el lado izquierdo
    let mut jugador = Pala {
        x: 40.0,
        y: screen_height() / 2.0 - 40.0,
        ancho: 15.0,
        alto: 80.0,
        velocidad: 6.0,
    };

    // Inicializamos la pelota en el centro exacto de la pantalla
    let mut pelota = Pelota {
        x: screen_width() / 2.0,
        y: screen_height() / 2.0,
        radio: 8.0,
        vel_x: 4.0,
        vel_y: 4.0,
    };

    } // Continuará en el Paso 4...

```

El programa ya se puede ejecutar sin dar ningún error pero se abre y se cierra inmediatamente porque no tenemos nada que visualizar.

Paso 4: El Corazón Asíncrono (El Game Loop)

Dentro de la función **main**, abrimos un bucle infinito **loop**. En cada vuelta (fotograma) ejecutaremos de forma secuencial las tres fases esenciales del desarrollo de videojuegos: Entradas, Lógica y Dibujo. Rematamos el bucle con un `.await` cooperativo.

El programa completo después del paso 4 es el siguiente:

```

use macroquad::prelude::*;

struct Pala {
    x: f32,
    y: f32,
    ancho: f32,
    alto: f32,
    velocidad: f32,
}

```

```

struct Pelota {
    x: f32,
    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}

#[macroquad::main("Rust Pong Educativo")]
async fn main() {
    // Inicializamos al jugador en el lado izquierdo
    let mut jugador = Pala {
        x: 40.0,
        y: screen_height() / 2.0 - 40.0,
        ancho: 15.0,
        alto: 80.0,
        velocidad: 6.0,
    };

    // Inicializamos la pelota en el centro exacto de la pantalla
    let mut pelota = Pelota {
        x: screen_width() / 2.0,
        y: screen_height() / 2.0,
        radio: 8.0,
        vel_x: 4.0,
        vel_y: 4.0,
    };

    loop {
        // --- 1. ENTRADAS (Leer Teclado) ---
        // Si el usuario pulsa Flecha Arriba y no se sale de la pantalla, sube
        if is_key_down(KeyCode::Up) && jugador.y > 0.0 {
            jugador.y -= jugador.velocidad;
        }
        // Si pulsa Flecha Abajo y no se sale por el suelo, baja
        if is_key_down(KeyCode::Down) && jugador.y < screen_height() -
jugador.alto {
            jugador.y += jugador.velocidad;
        }

        // --- 2. LÓGICA (Mover pelota y calcular colisiones) ---
        pelota.x += pelota.vel_x;
        pelota.y += pelota.vel_y;

        // Rebote en el techo y suelo
        if pelota.y - pelota.radio < 0.0 || pelota.y + pelota.radio >
screen_height() {
            pelota.vel_y = -pelota.vel_y; // Invierte la dirección vertical
        }

        // Rebote automático en la pared derecha (Para poder jugar solos)
        if pelota.x + pelota.radio > screen_width() {
            pelota.vel_x = -pelota.vel_x;
        }
    }
}

```

```

// Detección de colisión matemática con la pala del jugador
if pelota.x - pelota.radio <= jugador.x + jugador.anch
    && pelota.y >= jugador.y
    && pelota.y <= jugador.y + jugador.alto
{
    pelota.vel_x = -pelota.vel_x; // Rebota hacia la derecha
    pelota.x = jugador.x + jugador.anch + pelota.radio; // Antiatasco
}

// Reinicio si la pelota se escapa por la izquierda (Derrota)
if pelota.x < 0.0 {
    pelota.x = screen_width() / 2.0;
    pelota.y = screen_height() / 2.0;
    pelota.vel_x = 4.0;
}

// --- 3. DIBUJO (Pintar en pantalla) ---
clear_background(BLACK); // Limpia el frame anterior dejando el fondo
negro

// Dibujamos la pala del jugador (Blanca) y la pelota (Amarilla)
draw_rectangle(jugador.x, jugador.y, jugador.anch, jugador.alto,
WHITE);
draw_circle(pelota.x, pelota.y, pelota.radio, YELLOW);

// --- 4. LA PAUSA ASÍNCRONA ---
// Aquí cedemos el control al runtime de Macroquad. El hilo se
// pausa de forma segura para que la tarjeta gráfica dibuje la
// escena.
// En cuanto el monitor está listo para el siguiente frame,
// nuestra función se "despierta" sola.
next_frame().await;
}
}

```

¡Ya puedes ejecutar el programa con **cargo run** y jugar!

Cosas a resaltar

- **La linealidad protectora:** Fíjate que el código lee el teclado, luego calcula la posición de la pelota y finalmente dibuja. Al ejecutarse secuencialmente en un único hilo, es físicamente imposible que ocurra una condición de carrera. No necesitamos Arc ni Mutex porque ningún elemento compite por los datos.
- **El coste cero de la pausa:** La llamada a `next_frame().await` congela temporalmente la ejecución guardando el estado de nuestras estructuras jugador y pelota. No consume ciclos de CPU inútilmente mientras la pantalla se refresca, demostrando el potencial del modelo asíncrono frente al bloqueo tradicional de un `thread::sleep`.

6.1.1. Añadir un sistema de puntuación al código

Aquí tienes el listado completo del juego con el sistema de puntuación integrado. Hemos creado una estructura llamada **Marcador** para almacenar los puntos y utilizamos la función `draw_text` para renderizar el marcador en tiempo real en la parte superior de la pantalla. El

contador de puntos del jugador se incrementa cada vez que la pelota golpea con éxito la pala, y el de la IA / Fallos sube si la pelota se escapa por el lado izquierdo:

Proyecto: **juego_pong_puntuacion**

src/main.rs

```
use macroquad::prelude::*;

// =====
// 1. ESTRUCTURAS DE DATOS (Entidades del juego)
// =====

struct Pala {
    x: f32,
    y: f32,
    ancho: f32,
    alto: f32,
    velocidad: f32,
}

struct Pelota {
    x: f32,
    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}

struct Marcador {
    puntos_jugador: u32,
    fallos: u32,
}

// =====
// 2. FUNCIÓN PRINCIPAL ASÍNCRONA (Inicialización del Entorno)
// =====

#[macroquad::main("Rust Pong Educativo")]
async fn main() {
    // Inicializamos la pala del jugador en el extremo izquierdo
    let mut jugador = Pala {
        x: 40.0,
        y: screen_height() / 2.0 - 40.0,
        ancho: 15.0,
        alto: 80.0,
        velocidad: 6.0,
    };

    // Inicializamos la pelota en el centro geométrico de la pantalla
    let mut pelota = Pelota {
        x: screen_width() / 2.0,
        y: screen_height() / 2.0,
        radio: 8.0,
        vel_x: 4.0,
        vel_y: 4.0,
    };
}
```

```

// Inicializamos el sistema de puntuación en cero
let mut marcador = Marcador {
    puntos_jugador: 0,
    fallos: 0,
};

// =====
// 3. EL BUCLE DE JUEGO (Game Loop Asíncrono)
// =====
loop {
    // -----
    // FASE A: ENTRADAS (Control de teclado del Jugador)
    // -----
    if is_key_down(KeyCode::Up) && jugador.y > 0.0 {
        jugador.y -= jugador.velocidad;
    }
    if is_key_down(KeyCode::Down) && jugador.y < screen_height() -
jugador.alto {
        jugador.y += jugador.velocidad;
    }

    // -----
    // FASE B: LÓGICA (Físicas, Movimiento y Colisiones)
    // -----
    // Desplazamos la pelota según su velocidad actual
    pelota.x += pelota.vel_x;
    pelota.y += pelota.vel_y;

    // Rebote elástico en los límites superior e inferior de la ventana
    if pelota.y - pelota.radio < 0.0 || pelota.y + pelota.radio >
screen_height() {
        pelota.vel_y = -pelota.vel_y; // Invierte el vector vertical
    }

    // Rebote automático en la pared derecha (Simulación de rival estático)
    if pelota.x + pelota.radio > screen_width() {
        pelota.vel_x = -pelota.vel_x;
    }

    // Detección de colisión por caja delimitadora (AABB) con la pala del
Jugador
    if pelota.x - pelota.radio <= jugador.x + jugador.ancho
        && pelota.y >= jugador.y
        && pelota.y <= jugador.y + jugador.alto
    {
        pelota.vel_x = -pelota.vel_x; // Rebota hacia la derecha
        pelota.x = jugador.x + jugador.ancho + pelota.radio; // Corrige
intersección

        // ¡PUNTO LOGRADO! Incrementamos la puntuación del jugador por
salvar la pelota
        marcador.puntos_jugador += 1;
    }

    // Reinicio del punto si la pelota cruza el límite izquierdo (Punto

```

```

perdido)
    if pelota.x < 0.0 {
        pelota.x = screen_width() / 2.0;
        pelota.y = screen_height() / 2.0;
        pelota.vel_x = 4.0; // Restablece dirección inicial

        // ¡FALLO! El jugador no llegó a tiempo
        marcador.fallos += 1;
    }

    // -----
    // FASE C: RENDERIZADO (Dibujo Gráfico)
    // -----
    clear_background(BLACK); // Borra el fotograma anterior aplicando fondo
negro

    // Pintamos los elementos activos en la pantalla
    draw_rectangle(jugador.x, jugador.y, jugador.anchos, jugador.alto,
WHITE);
    draw_circle(pelota.x, pelota.y, pelota.radio, YELLOW);

    // --- RENDERIZADO DEL MARCADOR ---
    // Dibujamos los textos formateados en la parte superior central
    let texto_marcador = format!("Jugador: {} | Fallos: {}",
marcador.puntos_jugador, marcador.fallos);
    draw_text(&texto_marcador, screen_width() / 2.0 - 120.0, 40.0, 30.0,
GREEN);

    // Mensaje instruccional estático para el alumno
    draw_text("Controles: FLECHA ARRIBA / FLECHA ABAJO", 20.0,
screen_height() - 20.0, 20.0, GRAY);

    // -----
    // FASE D: SINCRONIZACIÓN ASÍNCRONA COOPERATIVA
    // -----
    next_frame().await;
}
}

```

Ahora podrás ver como sube el contador de puntos del jugador, que se incrementa cada vez que la pelota golpea con éxito la pala, y el de la IA que representa fallos del jugador.

6.1.2. Integr un canal mpvc y un hilo secundario para registrar los puntos en segundo plano

Aquí tienes el listado completo y definitivo del juego. Este código es la cumbre pedagógica de todo el Cuaderno 5, ya que fusiona en un solo archivo los dos grandes mundos que hemos estudiado: el **Game Loop asíncrono** (que corre a 60 FPS sin retrasos) y un hilo secundario concurrente conectado mediante canales MPVC para escribir los eventos en disco.

Cada vez que el jugador suma un punto o comete un fallo, el bucle asíncrono lanza un mensaje por el canal de forma instantánea. El hilo secundario capta el mensaje e imprime los datos, simulando una escritura pesada en disco sin provocar ni un solo tirón (lag) en la pantalla:

Proyecto: **juego_pong_canal_mvpc**

src/main.rs

```

use macroquad::prelude::*;
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

// =====
// 1. ESTRUCTURAS DE DATOS (Entidades del juego)
// =====

struct Pala {
    x: f32,
    y: f32,
    ancho: f32,
    alto: f32,
    velocidad: f32,
}

struct Pelota {
    x: f32,
    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}

struct Marcador {
    puntos_jugador: u32,
    fallos: u32,
}

// =====
// 2. FUNCIÓN PRINCIPAL ASÍNCRONA (Inicialización del Entorno)
// =====

#[macroquad::main("Rust Pong Educativo Completo")]
async fn main() {
    // -----
    // PARTE CONCURRENTE: CONFIGURACIÓN DEL CANAL MPSC E HILO SECUNDARIO
    // -----
    // Creamos el canal MPSC (Múltiples Productores, Un Solo Consumidor)
    let (tx, rx) = mpsc::channel::<String>();

    // Levantamos un hilo de ejecución secundario (nativo del S.O.)
    // Este hilo vivirá en paralelo al juego y se encargará del "trabajo sucio"
    thread::spawn(move || {
        // El bucle 'for' mantiene al hilo escuchando el receptor
        permanente
        for log in rx {
            // Simulamos que guardar este dato en el disco duro tarda un poco
            thread::sleep(Duration::from_millis(200));
            println!("[HILO SECUNDARIO - DISCO] Registrado con éxito: {}",
log);
        }
        println!("[HILO SECUNDARIO] Canal cerrado. El hilo termina de forma

```

```

limpia.");
});

// -----
// INICIALIZACIÓN DE ENTIDADES GRÁFICAS
// -----

let mut jugador = Pala {
    x: 40.0,
    y: screen_height() / 2.0 - 40.0,
    ancho: 15.0,
    alto: 80.0,
    velocidad: 6.0,
};

let mut pelota = Pelota {
    x: screen_width() / 2.0,
    y: screen_height() / 2.0,
    radio: 8.0,
    vel_x: 4.0,
    vel_y: 4.0,
};

let mut marcador = Marcador {
    puntos_jugador: 0,
    fallos: 0,
};

// =====
// 3. EL BUCLE DE JUEGO (Game Loop Asíncrono)
// =====
loop {
    // -----
    // FASE A: ENTRADAS (Control de teclado del Jugador)
    // -----
    if is_key_down(KeyCode::Up) && jugador.y > 0.0 {
        jugador.y -= jugador.velocidad;
    }
    if is_key_down(KeyCode::Down) && jugador.y < screen_height() -
jugador.alto {
        jugador.y += jugador.velocidad;
    }

    // -----
    // FASE B: LÓGICA (Físicas, Movimiento y Colisiones)
    // -----

    pelota.x += pelota.vel_x;
    pelota.y += pelota.vel_y;

    // Rebote elástico en los límites superior e inferior de la ventana
    if pelota.y - pelota.radio < 0.0 || pelota.y + pelota.radio >
screen_height() {
        pelota.vel_y = -pelota.vel_y;
    }

    // Rebote automático en la pared derecha
    if pelota.x + pelota.radio > screen_width() {

```

```

        pelota.vel_x = -pelota.vel_x;
    }

    // Detección de colisión con la pala del Jugador
    if pelota.x - pelota.radio <= jugador.x + jugador.ancho
        && pelota.y >= jugador.y
        && pelota.y <= jugador.y + jugador.alto
    {
        pelota.vel_x = -pelota.vel_x;
        pelota.x = jugador.x + jugador.ancho + pelota.radio;

        marcador.puntos_jugador += 1;

        // ¡CONEXIÓN ENTRE LOS DOS MUNDOS!
        // Enviamos un informe al hilo secundario. Como el método .send()
es instantáneo,
        // el juego no sufre ningún parón gráfico (anti-lag).
        let mensaje = format!("¡PUNTO! Marcador actual -> J:{} F:{}",
marcador.puntos_jugador, marcador.fallos);
        tx.send(mensaje).unwrap();
    }

    // Reinicio del punto si la pelota se escapa por la izquierda
    if pelota.x < 0.0 {
        pelota.x = screen_width() / 2.0;
        pelota.y = screen_height() / 2.0;
        pelota.vel_x = 4.0;

        marcador.fallos += 1;

        // Enviamos el aviso de fallo por el canal MPSC
        let mensaje = format!("¡FALLO! El jugador no llegó. Total fallos:
{}", marcador.fallos);
        tx.send(mensaje).unwrap();
    }

    // -----
    // FASE C: RENDERIZADO (Dibujo Gráfico)
    // -----
    clear_background(BLACK);

    draw_rectangle(jugador.x, jugador.y, jugador.ancho, jugador.alto,
WHITE);
    draw_circle(pelota.x, pelota.y, pelota.radio, YELLOW);

    // Renderizado del marcador en pantalla
    let texto_marcador = format!("Jugador: {} | Fallos: {}",
marcador.puntos_jugador, marcador.fallos);
    draw_text(&texto_marcador, screen_width() / 2.0 - 120.0, 40.0, 30.0,
GREEN);

    // Información de depuración educativa
    draw_text("El hilo secundario está registrando los eventos en
paralelo...", 20.0, screen_height() - 40.0, 18.0, DARKGRAY);
    draw_text("Controles: FLECHA ARRIBA / FLECHA ABAJO", 20.0,
screen_height() - 20.0, 18.0, GRAY);

```

```

// -----
// FASE D: SINCRONIZACIÓN ASÍNCRONA COOPERATIVA
// -----
next_frame().await;
    }
}

```

Resumen didáctico de la arquitectura de este código:

El Emisor (tx): Viaja dentro del bucle asíncrono (macroquad). Cada vez que la pelota colisiona, mete un texto en la tubería y sigue renderizando.

El Receptor (rx): Está en posesión exclusiva de un hilo nativo fuera del juego. Bloquea su propio flujo temporal ejecutando la espera lenta de `thread::sleep`, pero como pertenece a un núcleo distinto o un hilo independiente, la pantalla principal ni se entera.

Seguridad Absoluta: Rust garantiza mediante tipos que el canal se mantendrá íntegro y que el trasvase de hilos no provocará corrupciones de memoria.

6.2. Ejercicios propuestos

Para evaluar la asimilación del contenido, te proponemos modificar el código base añadiendo alguna de estas características:

- Incremento de dificultad: Hacer que la velocidad de la pelota aumente gradualmente cada vez que golpee la barra.
- Integración Web/Concurrente (Opcional): Usar hilos para guardar la puntuación más alta en un archivo en segundo plano sin congelar los gráficos del juego.

6.3. Desafío propuesto para el alumno (Personalización e integración)

Una vez que el motor base funciona de forma estable, puedes aplicar los conceptos avanzados estudiados a lo largo de los cuadernos de Rust para expandir las capacidades del videojuego.

Se propone el siguiente desafío:

- Enemigo Automatizado (IA Simple): Añade una segunda pala en el extremo derecho de la pantalla controlada por el juego. Puedes programar una lógica básica que compare la coordenada y de la pelota con la coordenada y de la pala enemiga para que esta se desplace hacia arriba o hacia abajo intentando interceptarla.

7. Servidor web asíncrono multihilo con Tokio

El ejemplo de servidor web multihilo que hemos elegido une de forma muy visual el mundo de la red, la asincronía y las tareas en segundo plano.

El código que proponemos más abajo utiliza un **modelo asíncrono multihilo** (work-stealing) gracias al **runtime de Tokio**.

🖥️ Abre una terminal integrada de VS Code en el directorio **proyectos-rust** creado al principio o en cualquier otro y crea un proyecto Rust con la orden: **cargo new servidor_web_multihilo**.

- Sitúate en la carpeta del proyecto mediante **cd servidor_web_multihilo** desde la misma terminal integrada de VS Code.
- Accede al fichero **Cargo.toml** y edita su sección [dependencies] para que se vea como se muestra abajo:

```
[dependencies]
axum = "0.8.9"
tokio = { version = "1.52.3", features = ["full"] }
```

- Copia el siguiente código y pégalo en el fichero src/main.rs

Fichero: **src/main.rs**

```
use axum::{routing::get, Router};
use std::net::SocketAddr;
use std::time::Duration; // Para pausar el tiempo

#[tokio::main]
async fn main() {
    // 1. Definimos la ruta de nuestra web
    let app = Router::new().route("/", get(leer_sensor));

    // 2. Configuramos la dirección de red
    let direccion = SocketAddr::from(([127, 0, 0, 1], 3000));
    println!("--- Servidor del Sensor Activo ---");
    println!("Pruébalo en este ordenador abriendo: http://localhost:3000");

    // Lanzamos la tarea cíclica de fondo (conteo 1 al 10)
    // Tokio se encarga de ejecutar esto de manera asíncrona en paralelo
    tokio::spawn(async {
        let mut contador = 1;
        loop {
            println!("Contador cíclico: {}", contador);
            contador += 1;
            if contador > 10 {
                contador = 1;
            }
            // Pausa asíncrona de 1 segundo sin bloquear el hilo
            tokio::time::sleep(Duration::from_secs(1)).await;
        }
    });
}
```

```

    }
});

// 3. Encendemos el servidor y lo dejamos escuchando.
// Nota: la última línea bloquea el final de main.
// Si no estuviera esa línea el programa se cerraría
let listener = tokio::net::TcpListener::bind(direccion).await.unwrap();
axum::serve(listener, app).await.unwrap();
}

// Función asíncrona que simula leer un sensor
async fn leer_sensor() -> String {
    // Simulamos que el sensor tarda 2 segundos en responder
    tokio::time::sleep(Duration::from_secs(2)).await;

    let temperatura = 24.5; // Un valor simulado
    format!("Temperatura actual del sensor: {} °C", temperatura)
}

```

- Ejecuta el programa mediante la orden **cargo run**.

El programa en su conjunto realiza varias tareas de forma asíncrona:

- El servidor está permanentemente escuchando en 127.0.0.1:3000
- Mediante `tokio::spawn(async { ... })` lanza el bloque asíncrono del contador que manda mensajes de forma continua a la pantalla.
- Cuando un cliente visita la ruta raíz("/") se ejecuta la función asíncrona `leer_sensor()` que devuelve información sobre temperatura.

7.1. Mecanismos clave que intervienen en el programa

1. El motor del sistema: tokio y tokio::main

La macro `#[tokio::main]` transforma la función `main` en una función asíncrona y levanta el runtime (entorno de ejecución) de **Tokio**.

Por defecto, Tokio configura un grupo de hilos nativos (Thread Pool) del sistema operativo igual al número de núcleos de la CPU.

El planificador (scheduler) se encarga de repartir las tareas de forma eficiente entre esos hilos. Si un hilo se queda sin trabajo, «roba» tareas de otros hilos (work-stealing) para maximizar el rendimiento.

2. El servidor web de alto rendimiento: axum

- **Router::new().route(«/», get(leer_sensor))**: Axum aprovecha el ecosistema asíncrono para gestionar las peticiones HTTP. Cuando llega una solicitud a la raíz (/), Axum no bloquea un hilo esperando a que se procese; simplemente genera una tarea asíncrona (Future).
- Al usar **axum::serve**, el servidor se queda escuchando de forma no bloqueante. Puede recibir miles de conexiones simultáneas en un solo hilo del sistema porque la espera se gestiona a nivel de eventos de red.

3. La creación del «hilo de fondo»: tokio::spawn

- Es vital aclarar que **tokio::spawn** no crea un hilo del sistema operativo. Lo que crea es una Green Thread (o **tarea asíncrona**).
- Las tareas asíncronas en Rust son increíblemente ligeras (pesan apenas unos bytes o kilobytes en memoria, a diferencia de los megabytes que requiere un hilo real del sistema).
- Tokio toma ese bloque **async { ... }** y lo pone en la cola del planificador. Este contador cíclico se ejecutará en paralelo con el servidor web, saltando de un hilo a otro del pool de Tokio según haga falta, de forma transparente para el desarrollador.

4. La magia del no-bloqueo: **tokio::time::sleep(...).await**

- En el bucle del contador usamos **tokio::time::sleep**, y en la función del sensor hacemos lo mismo. La palabra clave **.await** es el «punto de rendimiento» voluntario.
- Cuando el programa llega a esa línea, la tarea le dice al planificador de Tokio: «Voy a estar esperando 1 o 2 segundos. Retírame de la CPU y aprovecha este hilo para atender peticiones web o avanzar con otra tarea».
- Una vez cumplido el tiempo, Tokio despierta la tarea y la vuelve a poner en la cola para continuar justo donde se quedó. Si usaras **std::thread::sleep**, congelarías el hilo entero del sistema operativo, impidiendo que otras tareas avancen.

7.2. ¿Cómo reparte Tokio las tareas entre los hilos? (El Planificador)

El planificador de Tokio de tipo Work-Stealing (Robo de Trabajo) funciona como un director de orquesta ultraeficiente. Imagina un procesador con 4 núcleos (Hilo 1 al Hilo 4). Cuando arrancas el servidor, Tokio asigna una cola local de tareas a cada hilo del sistema operativo. El reparto visual se comporta así:

```
HILO 1 del Sistema —> Cola Local —> 📋 Tarea: Servidor Axum (Escuchando)
HILO 2 del Sistema —> Cola Local —> 📋 Tarea: Contador Cíclico (tokio::spawn)
HILO 3 del Sistema —> Cola Local —> (Vacía - Buscando trabajo)
HILO 4 del Sistema —> Cola Local —> (Vacía - Buscando trabajo)
```

El flujo de ejecución paso a paso:

1. Lanzamiento: Al ejecutar **tokio::spawn**, el Contador Cíclico se envía al planificador. Este lo coloca en la cola de uno de los hilos libres (por ejemplo, el Hilo 2).
2. Petición Web: Cuando un usuario entra a <http://localhost:3000>, el Servidor Axum (en el Hilo 1) recibe la conexión y genera una nueva tarea: 📋 Tarea: leer_sensor().
3. El Punto **.await** (Cooperación): En cuanto la tarea **leer_sensor()** llega a su **tokio::time::sleep(...).await**, libera el Hilo 1. La tarea se pausa en segundo plano esperando el temporizador, dejando el Hilo 1 completamente vacío y listo para recibir a otro usuario.
4. Robo de Trabajo (Work-Stealing): Si el Hilo 3 o el Hilo 4 se quedan sin tareas en su cola local, no se duermen. Miran las colas de los Hilos 1 y 2 y «roban» de forma segura cualquier tarea que esté lista para ejecutarse (como el contador cíclico cuando despierta de su segundo de pausa).

7.2.1. Diferencia de Consumo de Memoria: `std::thread` vs `tokio::spawn`

Para entender por qué la asincronía escala tan bien, debemos comparar qué le cuesta al ordenador crear un hilo real frente a una tarea asíncrona.

Analicemos la siguiente tabla:

Característica	Hilo Nativo (<code>#raw(«std::thread»)</code>)	Tarea de Tokio (<code>#raw(«tokio::spawn»)</code>)
Nivel	Administrado por el Sistema Operativo .	Administrado por el Runtime de Tokio (en espacio de usuario).
Tamaño en Memoria	¡De 1 a 2 Megabytes (MB)! (Reserva fija para la pila/stack).	Unos pocos Kilobytes (KB) o incluso Bytes (dinámico según variables).
Límite Práctico	Unos pocos miles (el sistema se colapsa).	Millones de tareas simultáneas sin despeinarse.
Cambio de Contexto	Costoso . Involucra al kernel de la CPU para salvar registros de hardware.	Barato . Es un simple cambio de puntero y llamada a una función (<code>poll</code>).

Cuando creas un hilo nativo con `std::thread`, el sistema operativo es precavido y le asigna un «colchón» de memoria enorme (normalmente 2 MB en Linux) por si la función realiza muchas llamadas internas. Si intentas abrir 50.000 hilos nativos, tu servidor consumirá unos 100 GB de RAM solo en existir, lo que tumbará la máquina.

En cambio, `tokio::spawn` no pide memoria fija al sistema operativo. En Rust, un bloque asíncrono se compila como una Máquina de Estados (una estructura de datos interna). Solo ocupa el espacio estrictamente necesario para guardar las variables que están activas durante el `.await`.

- La tarea del contador cíclico solo guarda un entero (contador). ¡Ocupa prácticamente nada en memoria! Puedes levantar 1.000.000 de contadores asíncronos y el servidor apenas consumirá unas pocas decenas de Megabytes.

Hemos preparado dos retos finales para animar a arremangarte y modificar el programa anterior.

8. Los retos propuestos

A continuación de proponemos dos retos que hemos denominado **Reto Final A** y **Reto Final B**. Para llevarlos a cabo necesitarás información que no está recogida en el cuaderno. Te daremos pistas pero el objetivo es que consigas lo necesario a través de la web, preguntando a Google o directamente a una IA para completar los retos.

8.1. 🏆 Reto Final A: El Panel de Control del Operador

¡Ha llegado el momento de poner a prueba tus nuevos superpoderes asíncronos! Actualmente, nuestro servidor web lee un sensor simulado estático y, por otro lado, un contador cíclico imprime números en la consola de forma independiente. Tu misión, si decides aceptarla, es unirlos.

🎯 El Objetivo

Modifica el código para que el contador cíclico actúe como un «generador de telemetría» que actualiza el valor del sensor en tiempo real. Cuando el usuario visite <http://localhost:3000>, el servidor web debe responder mostrando el último valor que ha calculado el contador de fondo.

💡 Pistas para resolverlo:

Para lograr que la tarea de fondo (`tokio::spawn`) y la función del servidor web (`leer_sensor`) compartan datos de forma segura en un entorno multihilo, necesitarás usar las herramientas de sincronización que nos da Rust:

1. El Estado Compartido (Arc + Mutex): No puedes usar variables globales normales porque Rust te protegerá de carreras de datos (data races). Envuelve el contador en un `Arc::new(Mutex::new(0))` para poder compartirlo de forma segura entre hilos.
2. Pasar el Estado a Axum: Axum permite pasar datos a las rutas usando `.with_state()`. Investiga cómo recibir ese estado en la función `leer_sensor` usando el extractor `State`.
3. ¡Cuidado con los Mutex del sistema! Recuerda que estás en un entorno asíncrono. Si usas un Mutex tradicional de la librería estándar (`std::sync::Mutex`) y lo bloqueas a través de un `.await`, podrías congelar el planificador de Tokio. Utiliza el Mutex asíncrono de Tokio: `tokio::sync::Mutex`.

El Resultado Esperado

Al abrir tu navegador y refrescar la página varias veces, verás cómo la temperatura cambia dinámicamente según el segundo exacto en el que hagas la petición:

- Intento 1: Temperatura actual del sensor: 24.1 °C
- Intento 2 (un segundo después): Temperatura actual del sensor: 24.2 °C

8.2. 🏆 Reto Final B: El Sistema de Alertas Inteligente

Ahora que entiendes cómo Tokio reparte el trabajo y cómo Axum atiende a tus usuarios, es hora de poner a prueba tus habilidades. Vamos a transformar nuestro servidor estático en un sistema de monitoreo en tiempo real.

🎯 El Objetivo

Modifica el código actual del cuaderno para cumplir con las siguientes directrices:

1. La Variable Compartida: Crea una variable global (o compartida de forma segura) que almacene la temperatura actual del sensor.
2. El Hilo de Fondo Actualiza: En lugar de solo imprimir del 1 al 10, haz que la tarea de fondo (`tokio::spawn`) simule la lectura del sensor generando una temperatura aleatoria cada 2 segundos y actualice la variable compartida.

3. El Servidor Responde al Instante: Modifica la función `leer_sensor` de Axum para que, cuando el usuario entre a <http://localhost:3000>, ya no espere 2 segundos de forma fija, sino que responda inmediatamente mostrando el último valor que haya calculado la tarea de fondo

Pistas para resolverlo

- El problema de compartir en hilos: Como Tokio puede mover tus tareas entre diferentes hilos del sistema operativo, no puedes usar una variable común y corriente. Rust te obligará a protegerla.
- Tus herramientas aliadas: Necesitarás combinar un puntero inteligente de pertenencia compartida (`std::sync::Arc`) con un cerrojo de exclusión mutua asíncrono (`tokio::sync::Mutex`) para poder leer y escribir la temperatura de forma segura entre la tarea del contador y la ruta de Axum.
- Para los datos aleatorios: Puedes usar la funcionalidad de tiempo de Rust para **inventar un número** o añadir la dependencia **rand** en tu Cargo.toml.

Pista de código (Estructura base)

Por si te atascas te dejamos esta pequeña plantilla de cómo se inicializa el estado compartido en Axum:

```
// Ejemplo de cómo compartir el estado en Axum usando Arc y Mutex
use std::sync::Arc;
use tokio::sync::Mutex;
use axum::{Extension, Router, routing::get};

// 1. Creamos un tipo para nuestro estado
type EstadoCompartido = Arc<Mutex<f32>>;

#[tokio::main]
async fn main() {
    // 2. Inicializamos la temperatura en 25.0 grados
    let temperatura_compartida: EstadoCompartido = Arc::new(Mutex::new(25.0));

    // 3. Pasamos el estado a Axum usando "Extension"
    let app = Router::new()
        .route("/", get(leer_sensor))
        .layer(Extension(temperatura_compartida.clone()));

    // ... El resto de tu código de Tokio ...
}
```